Correct Values Can Be Recovered Before Wrong Reasoning Steps

Anonymous ACL submission

Abstract

We investigate whether correct values remain recoverable from a language model's internal activity when misleading names lead it to report wrong calculations. Building on Kudo et al. (2026), we change variable names in arithmetic problems and Python programs while keeping their computations fixed. In code, a name suggesting summation can make a model report the sum of a list where the program computes its length. On these incorrect intermediate outputs in three tested code models, a separately trained predictor recovers the correct digit with 95.5–99.7% accuracy immediately before the model reports the wrong digit. Showing the operation alongside its result in worked examples improves both intermediate and final correctness. On the same misleading-name programs, OLMo-2-7B's final-answer accuracy rises from 37.1% to 92.6%. Examples that explicitly connect each value to its computation offer a practical way to improve written reasoning.

1 Introduction

Written calculations let readers check how a language model reaches an answer. But the values in those calculations can follow a misleading variable name instead of the operation in the problem. Consider a Python program that stores `len([5, 3])` in a variable named `sum_all`. The correct value is 2, but the name suggests 8. In Figure 1, OLMo-2-7B writes `sum_all = 8` and ultimately answers 15 instead of 9. The figure compares the same program under prompts whose worked examples show only values or show how each value is calculated. We investigate whether the correct value is still recoverable immediately before such a wrong write, and whether changing the worked examples in the prompt improves what the model writes.

Kudo et al. (2026) track intermediate arithmetic values in model activity during generated calculations. We build on their controlled arithmetic task and methods to study a conflict between a variable's name and its computed value, then extend the investigation to short Python programs. Each problem has two versions with identical computations and different names. This comparison measures whether a misleading name increases the specific wrong answer it suggests. Written arithmetic calculations show small name effects in most tested conditions; code provides frequent intermediate errors on which to examine the gap between internal activity and written output.

In code, that gap is pronounced. When worked examples list intermediate values alone, misleading names frequently lead the model to write the suggested wrong value. Yet a separately trained predictor recovers the correct digit from model activity with 95.5–99.7% accuracy immediately before those errors. Showing the source operation alongside its result in the worked examples sharply reduces wrong writes and increases correct intermediate and final outputs (Table 2). The improvement also holds with fresh names and new programs.

The results connect a failure in written reasoning to a practical way to improve it. Correct-value recovery and correct written steps are distinct measurements, and the content of worked examples affects the latter. Evaluations should check both intermediate values and final answers, while examples that show how each value is computed offer a concrete way to improve their reliability.

2 Experimental design

2.1 Inputs and requested outputs

Arithmetic problems contain single-digit addition and subtraction, following Kudo et al. (2026). In the main level, the queried equation precedes the one needed to evaluate it. For `pen=1+cup, cup=2+3; pen=?`, the answer is 6. Code prob-

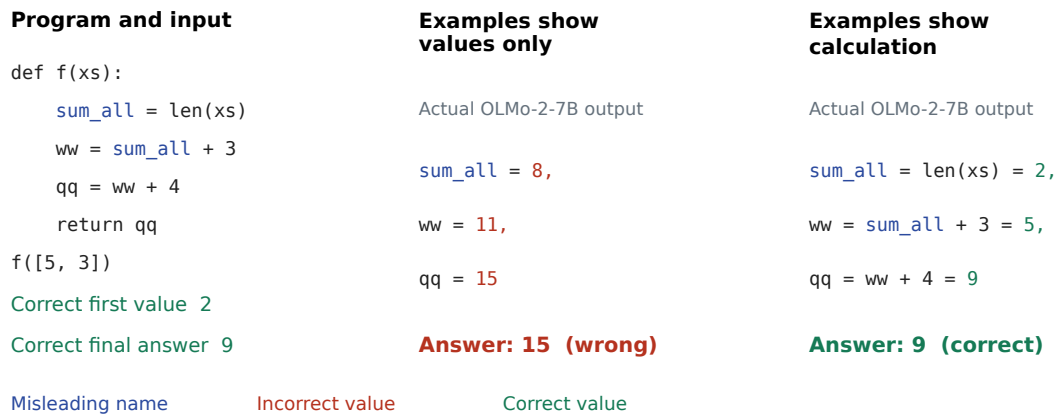


Figure 1: OLMo-2-7B outputs for the same program with different worked examples in the prompt.

lems contain a short Python function and its input list (Figure 1). Assignments compute length, sum, maximum or minimum, then integer arithmetic. The main level has three dependent assignments; all correct assignment values and return values are single digits.

The arithmetic evaluation covers thirteen checkpoints, principally at level 3, with 2,000 matched problems per tested level. The code evaluation covers seven instruction-tuned checkpoints at levels 3 and 5, with 1,000 matched problems per target and level, plus a CodeLlama-34B check. The central recovery analysis examines OLMo-2-7B, Llama-3.2-3B and Llama-3.1-8B, three instruction-tuned models with substantial length-to-sum errors, on the 285-program level-5 subset. Arithmetic and code use different checkpoint and prompt panels; their results are reported separately. Complete model lists and task levels remain in Appendices B and J.

Three worked examples precede each test problem. In arithmetic, the model either writes only the answer or evaluates the equations before answering. In code, it writes each assignment's value and then the function's return value. We compare examples showing values alone, such as `v=2`, with examples showing the source expression and value, such as `v=len(xs)=2`. The test program and `Trace:` prefix are identical across these formats. Generation is greedy; three fixed demonstration sets test sensitivity to the examples. Complete prompt formats are in Appendix A.

2.2 Matched names and output scoring

Each problem has an ordinary-name twin and a misleading-name version. Inputs, operations and correct values stay fixed. In arithmetic, renaming `pen` to `four` suggests 4 instead of 6. In code, `sum_all` suggests the sum of a list where the program computes its length. The suggested digit differs from the correct value and the input digits. This supplies a specific competing wrong answer whose rate can be compared between the twins.

Table 1 defines the output measurements. The main code error measurement concerns the first assignment; arithmetic name errors concern the final answer. We report accuracy alongside these errors to measure whether outputs become correct. Parse failures remain in all behavioral denominators and count as incorrect for accuracy. Appendix A gives the parsing rules; Appendix X gives the matched code correctness cohort.

2.3 Measuring internal activity

A *linear probe* is a separately trained predictor of the correct digit from one hidden state. In the central code analysis, it reads the state at the `=` immediately before an actual wrong value. We also read the end of the prompt, before the trace starts, on those same error cases. Probe training uses ordinary-name programs whose computations are disjoint from evaluation. Layer selection excludes the held-out computation, and the cached prefix must match the generated prefix through the measurement position. The answer token is excluded.

A probe measures recoverable information; it does not establish that generation causally uses that information. The language model generates without the probe. *Activation patching* instead replaces activity with the ordinary-name twin's activity and measures the resulting output change. Arithmetic readouts mainly use supplied correct

Measurement	What counts
Intermediate correctness	The written value matches the executed assignment value.
Final correctness	The reported answer matches the correct result.
Name-suggested error	The target output equals the specific wrong digit suggested by the name.
Excess name errors	Misleading-name rate minus ordinary-name rate for that same wrong digit.

Table 1: Output measurements. Intermediate correctness is reported for the first code assignment.

calculations, with freely generated calculations tested separately. Probe and patching details are in Appendices D.1, N and V.

2.4 Uncertainty and controls

Intervals resample computations, keeping both naming conditions and repeated demonstrations together. A reliable effect requires the same sign across the three demonstration sets and a 95% interval excluding zero. A small-effect claim requires a 90% interval inside ± 2 percentage points. Intervals are pointwise and condition on fitted layer selectors. Models and formats remain separate comparisons. Appendix P and Appendix A give the sampling and inference details.

3 Misleading names can change written steps

3.1 Code traces can follow the name's suggested operation

Misleading names produce substantial wrong intermediate writes when worked examples list values alone. In length-computing programs, they add +58.4, +43.4 and +12.4 percentage points of sum writes in OLMo-2-7B, Llama-3.2-3B and Llama-3.1-8B, respectively. All three checkpoints are instruction-tuned. These are effects on the first assignment's written value, not just on the final answer.

The sum is not a common ordinary-name mistake at this step. It accounts for only 14–32% of parsed incorrect ordinary-name writes; maximum values and off-by-one lengths are more common. On the misleading-name programs, correct first writes fall to 37.4%, 51.5% and 87.6% (Table 2). The matched comparison thus identifies a specific competing calculation that enters the written trace.

3.2 Arithmetic provides a comparison with fewer name errors

Misleading-name effects are generally small when arithmetic calculations are written. In 43 of 44 comparisons, the added-error interval fits within two points of zero. Ten comparisons show a reliable reduction relative to answer-only output; many answer-only effects are already small (Appendix H). The full behavioral panel and selected contrasts are in Appendix Table 6 and Figure 8.

Some output failures remain possible after correct calculations. With an entire correct generated calculation held fixed, OLMo-2-1B has 12.8 added-error points when the final query omits the variable name. Repeating the name reduces the estimate to 0.5 points, while changing ordinary-name accuracy. These arithmetic comparisons motivate checking outputs even after correct intermediate work. They provide fewer errors than code for the central readout test; calibration and competence differences are consolidated in Section 6 and Appendix I.

4 Correct values are recoverable before wrong steps

4.1 Read the state immediately before the observed failure

Figure 2 asks whether the correct value can be predicted from internal activity on cases where the model reports the sum instead of the length. Gray squares measure predictor accuracy before the model starts writing; green circles measure it just before the incorrect value. Both use the same observed errors. The green points reach 95.5–99.7% accuracy immediately before the wrong value is generated. The figure reports the number of unique error-producing programs for each model. Multiple demonstration sets can produce errors on the same program; those repeats remain clustered.

Correct digits are much more readily recovered just before the wrong value than before the trace starts. Both positions use independent layer selection on the same error-producing computations. The next written digit is therefore wrong at a point where the predictor reliably recovers the correct digit. Appendix V reports the paired changes and intervals.

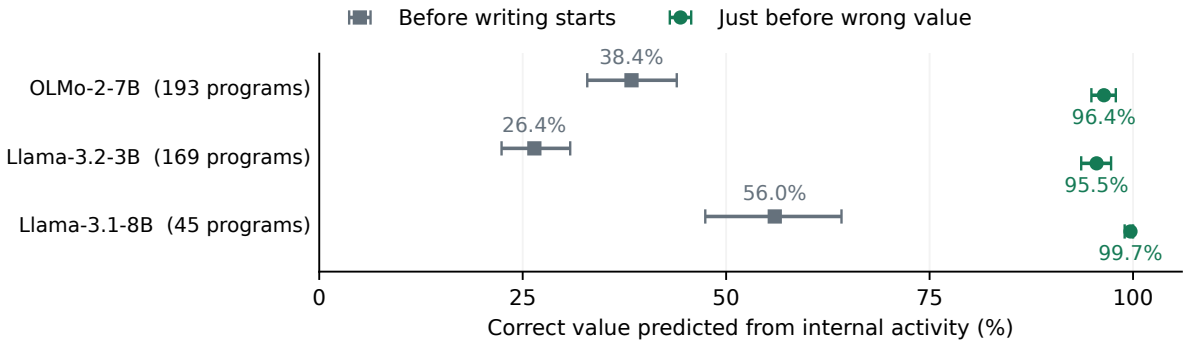


Figure 2: Predictor accuracy on the same eventual errors at both positions (95% intervals). Counts are unique programs.

The predictors train on 2,000 separate ordinary-name programs, and select layers without using the held-out computation. The prefix audit matches the cached state to the actual generation through the = in all 2,565 checked generations. These checks exclude computation overlap, layer selection on the tested error, and access to the future digit.

Intervention evidence. Replacing name-token activity with the ordinary-name twin's activity removes 97% and 100% of name errors in the two code models with aligned name sites. All-layer pre-value patches remove 97–100%; the ordinary-word control changes at most 1.0% of correct outputs. Error removal can yield another wrong value; these whole-state patches test output sensitivity rather than the decoded digit alone (Appendices N and D).

5 Showing the operation reduces written errors

5.1 Change the demonstrations, then inspect the same programs

The strongest practical comparison keeps the program, input, model and demonstration identities fixed. Replacing `v=2` examples with `v=len(xs)=2` connects the reported value to the source operation. Both first-assignment and final-answer accuracy improve (Table 2). On the same misleading-name programs, final-answer accuracy rises from 37.1% to 92.6% in OLMo-2-7B and from 51.0% to 99.4% in Llama-3.2-3B.

Excess name errors identify the specific naming effect. Added sum writes fall to +3.4, +0.4 and +0.0 points in OLMo-2-7B, Llama-3.2-3B and Llama-3.1-8B, respectively. Figure 3 compares four ways to write the worked examples. Its horizontal axis measures extra sum errors under misleading names, relative to ordinary names. Zero means the name does not increase that error; farther right means a larger increase.

Ordinary-name correctness remains high; Appendix X gives both naming conditions and paired format changes. Each comparison uses 855 paired observations across 285 programs.

Adding words such as "checked" leaves large name effects. Adding arithmetic such as `v=2+0=2` helps partly. Showing the program's actual calculation, such as `v=len(xs)=2`, gives the largest reduction among these controls. The added-text examples match the calculation examples in token count; the extra-arithmetic examples are a separate control. The test program and trace prefix are unchanged. The benefit therefore depends on what the added text contains. Paired changes and intervals are in Appendix S.

5.2 The format contrast replicates with fresh names

The original code design planned to retain names that evoke their intended operation in at least 80% of models, but the runs retained the full name table. To test the format contrast with independently selected names, we screen 16 fresh sum-family candidates by their highest-scoring operation continuation before testing traces. Of these, 14 pass the 80% panel criterion. We freeze the first 3 qualifying names and cross them with 200 new length-computing programs. The computations are absent from the original training and test data and every demonstration set.

With values-only examples, added sum writes are 91.1, 61.5 and 16.7 points in OLMo-2-7B and

Model	Correct first value (%) Values only → calculation shown	Correct final answer (%) Values only → calculation shown
OLMo-2-7B	37.4 → 92.0	37.1 → 92.6
Llama-3.2-3B	51.5 → 99.3	51.0 → 99.4
Llama-3.1-8B	87.6 → 100.0	87.7 → 100.0

Table 2: Correct outputs on the same misleading-name programs under both example formats.

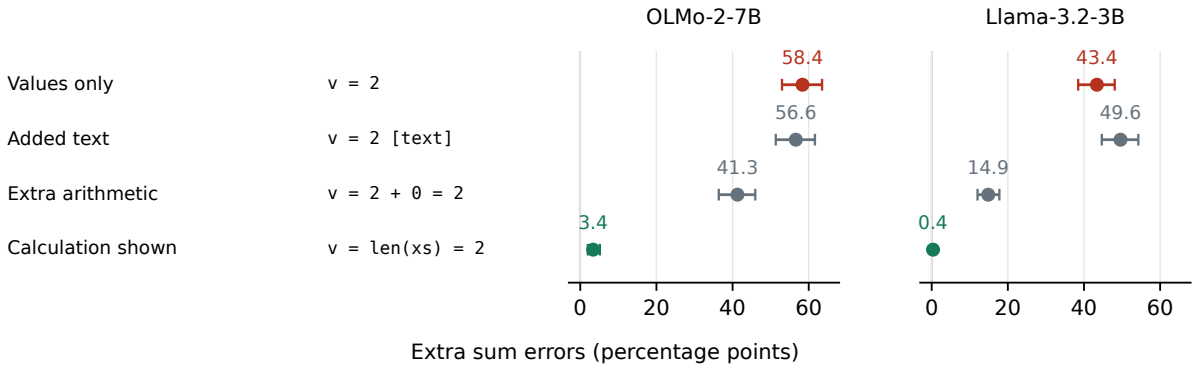


Figure 3: Sum errors report a list's sum instead of its length. Rates subtract the ordinary-name baseline; bars show 95% intervals. [text]: added filler words.

the two Llama models. Examples showing the calculation reduce these to 0.8 points in OLMo and zero observed excess in both Llamas. The prespecified CodeGemma-7B control has zero observed excess in both formats. Figure 4 uses the same error measure as Figure 3. Each row is a new sum-suggesting variable name, and each panel is a model. Red circles show values-only examples; green squares show examples with the calculation. Their separation in the three affected models shows that the improvement holds across all three names. Intervals cluster repeated demonstrations of each computation; pooled intervals also cluster names (Appendix U). This replication tests the format contrast on new names and programs. The internal readouts and patches concern the original cohort.

6 Scope of the findings

Models and formats. The discrepancy is strongest in three code checkpoints under values-only demonstrations. Other models and operation pairings show different effects (Appendix L). Interactive Python transcripts and values written as code comments also reduce name errors (Appendix Table 28); source expressions are one effective format. The reported direct-answer contrasts rename the returned variable, whereas the central trace comparison renames the first assignment. Those contrasts do not isolate an effect of requesting a trace.

Arithmetic readouts. Recovery on actual arithmetic errors is harder to assess. Predictors trained on supplied correct calculations do not reliably recover values from OLMo's freely generated calculations, while the Llama models make too few errors for the same strong test as code. Under supplied correct calculations, the probes reproduce the progression of recoverable values studied by Kudo et al. Calibration criteria and detailed comparisons are in Appendices G and E.

Arithmetic demonstration content. Some format changes also change accuracy on ordinary-name arithmetic problems. Those comparisons cannot cleanly attribute a reduction in name errors to the presence of equations, because general task accuracy changes too. In the tested Llama models, equations with and without variable names both reach 99–100% ordinary-name accuracy and near-zero excess name errors. Repeating names is therefore unnecessary in those formats. The accuracy changes, comparison criteria and qualifying counts are in Appendix I.

Natural code and scale. The CRUXEval check (Gu et al., 2024) did not establish a clear accuracy difference between ordinary and misleading names under its tested formats. It covers 89 functions, of which 84 use the renamed vari-

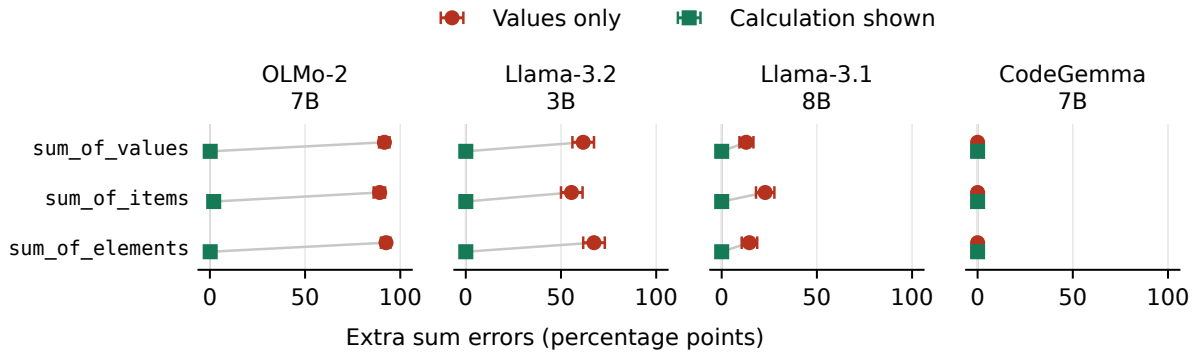


Figure 4: Three new names for list-length variables on 200 new programs. Extra sum errors are relative to ordinary names; bars show 95% intervals.

able downstream; accuracy-difference intervals include zero. This check requests direct answers or prose explanations, rather than the assignment-by-assignment trace used in the synthetic experiments. It therefore tests a different output setting and does not directly test the central trace finding. Separately, CodeLlama-34B's maximum-to-sum excess is +1.8 points [+0.7, +3.1]. Its interval extends beyond two points, so this estimate does not establish an effect within the small-effect margin (Appendix O).

7 Related work

Computation during reasoning. Kudo et al. (2026) track arithmetic values with probes and activation interventions, finding that answers develop during the calculation. We retain their controlled arithmetic and probe recipe, then introduce a name with a competing answer and extend the conflict to programs. Our code timing comparison concerns the same observed wrong writes before and during a trace. Work on positional readouts and shortcut solutions also motivates testing what a readout establishes (Liu, 2026; Prabhakar et al., 2024).

Names and misleading wording. Identifier changes affect code comprehension and robustness (Gao et al., 2023; Le et al., 2025; Lam et al., 2025; Le et al., 2026; Orvalho and Kwiatkowska, 2025). Our tasks provide an executable correct value and a distinct value suggested by the name. The matched comparison measures which incorrect value changes, rather than attributing every accuracy loss to the same influence. Stroop and cue-conflict studies supply a related experimental logic (Stroop, 1935; Geirhos et al., 2019; Wang, 2026; Hu et al., 2026); the analogy does not establish a shared human and model mechanism.

Written reasoning and internal information. Faithfulness studies show that explanations can omit influences on outputs or react differently to perturbations (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025). Orgad et al. (2025) show that internal representations can identify correct answers among generated candidates even when the model tends to answer incorrectly. Our contribution measures recovery before an incorrect *intermediate* write, with the answer token excluded and a controlled name suggesting a particular competing value. We also test a demonstration-format change that improves intermediate and final correctness on the same programs. Probe controls and activation interventions assess recoverability and output sensitivity separately (Hewitt and Liang, 2019; Zhang and Nanda, 2024).

8 Discussion and conclusion

The central finding is a discrepancy at the point of writing. Correct digits are recoverable immediately before a model writes conflicting digits suggested by variable names. Demonstrations showing the producing operation sharply reduce those errors and increase correct outputs. These observations have three implications.

Interpreting internal measurements. Successful decoding establishes recoverable information, but neither guarantees that the model will write it nor proves that generation causally uses it. The code error cohorts demonstrate this distinction directly. For a length-computing program, the predictor may recover list size or related input features; recovery does not identify the internal com-

putation that produced that information. More selective interventions are needed to test the decoded value's role in generation.

Evaluating written reasoning. Internal recoverability, intermediate correctness and final-answer correctness are distinct properties. In code, the correct intermediate value is recoverable immediately before an incorrect one is written. In arithmetic, even a correct written calculation can be followed by a name-suggested wrong final answer. A final-answer score alone leaves the intermediate values unmeasured, while a successful probe cannot certify either written steps or final answers. Each measurement answers a different question about reasoning reliability.

Designing demonstrations. What each written step contains matters. Source-operation examples improve the tested code traces, including on fresh names and programs. Matched additional text leaves large effects; numeric elaboration reproduces only part of the benefit. The next test is transfer to larger programs and natural explanations with adequate ordinary-name accuracy. The current evidence establishes an effective format change and the gap it reduces, without identifying a complete causal explanation of that gap.

Limitations

- **Tasks and formats.** Inputs are synthetic arithmetic and short programs with single-digit values. The largest code effects use a values-only trace; interactive Python and comment formats show smaller effects. A 89-function CRUXEval check tests direct-answer and prose accuracy, not the synthetic trace. Its intervals cover zero and do not establish equivalence within the two-point margin. Models reach 34B, but the arithmetic and code panels differ in checkpoints and do not cover all model families.

- **Readouts and interventions.** The central code probes cover three models. For length-computing programs, a probe may recover the number of list items or related input features; it does not identify how that value was computed inside the model. Arithmetic probes are unreliable on OLMo's freely written calculations, and the Llama models make too few errors for the same strong test. Patches replace mixed representations and may produce another wrong answer. Neither the probes nor the patches identify one mechanism shared across the tasks.

- **Selection and competence.** The original code identifiers depart from the planned meaning gate; the independently screened replication tests new names and computations. Accuracy changes accompany several arithmetic format controls. Conditional correct-calculation cohorts do not estimate the effect of making a calculation correct. Intervals condition on fitted selectors and the tested demonstrations; related checkpoints and repeated roles are not independent models.

References

Akhiad Bercovich et al. 2025. Llama-Nemotron: Efficient reasoning models. *arXiv preprint arXiv:2505.00949*.

Yanda Chen et al. 2025. Reasoning models don't always say what they think. *arXiv preprint arXiv:2505.05410*.

CodeGemma Team. 2024. CodeGemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409*.

Shuzheng Gao, Cuiyun Gao, Chaozheng Wang, Jun Sun, David Lo, and Yue Yu. 2023. Two sides of the same coin: Exploiting the impact of identifiers in neural code comprehension. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, pages 1933–1945. ArXiv:2207.11104.

Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.

Gemma Team. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.

Aaron Grattafiori et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Alex Gu, Baptiste Rozière, Hugh James Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida Wang. 2024. CRUXEval: A benchmark for code reasoning, understanding and execution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16568–16621. PMLR.

John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743.

Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan Huang, Aman Taxali, Alex Weigard, Richard L. Lewis, and Chandra Sripada. 2026. Conflict and congruency effects in large language models: In-weight and in-context competition in a verbal conflict task. *arXiv preprint arXiv:2608.11510*.

Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi, Shusaku Sone, Masaya Taniguchi, Ana Brassard, Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs faithfully and iteratively compute answers during CoT: A systematic analysis with multi-step arithmetics. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 1114–1153.

Man Ho Lam, Chaozheng Wang, Jen-Tse Huang, and Michael R. Lyu. 2025. CodeCrash: Exposing LLM fragility to misleading natural language in code reasoning. In *Advances in Neural Information Processing Systems*, volume 38, pages 120782–120828.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamilė Lukošiūtė, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*.

Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van, Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen. 2025. When names disappear: Revealing what LLMs actually understand about code. *arXiv preprint arXiv:2510.03178*.

Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026. Do machines struggle where humans do? LLM and human comprehension of obfuscated code. *arXiv preprint arXiv:2606.31725*.

Ming Liu. 2026. The readout shortcut: Positional number copying dominates arithmetic CoT readout in small language models. *arXiv preprint arXiv:2605.22870*.

Hadas Orgad, Michael Toker, Zorik Gekhman, Roi Reichart, Idan Szpektor, Hadas Kotek, and Yonatan Belinkov. 2025. LLMs know more than they show: On the intrinsic representation of LLM hallucinations. In *The Thirteenth International Conference on Learning Representations*.

Pedro Orvalho and Marta Kwiatkowska. 2025. Are large language models robust in understanding code against semantics-preserving mutations? *arXiv preprint arXiv:2505.10443*.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830.

Akshara Prabhakar, Thomas L. Griffiths, and R. Thomas McCoy. 2024. Deciphering the factors influencing the efficacy of chain-of-thought: Probability, memorization, and noisy reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3710–3724.

Baptiste Rozière et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Skipper Seabold and Josef Perktold. 2010. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96.

J. Ridley Stroop. 1935. Studies of interference in serial verbal reactions. *Journal of Experimental Psychology*, 18(6):643–662.

Team OLMo. 2024. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, pages 74952–74965.

Evan Pete Walsh et al. 2025. 2 OLMo 2 furious (COLM's version). In *Conference on Language Modeling*.

Han-yu Wang. 2026. Persistent priors, preserved targets: A Stroop-style paradigm for lexical override. *arXiv preprint arXiv:2606.07555*.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on*

Empirical Methods in Natural Language Processing: System Demonstrations, pages 38–45, Online. Association for Computational Linguistics.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *International Conference on Learning Representations*.

A Prompts and output scoring

Arithmetic prompts. The main few-shot prompt concatenates three same-level ordinary-name demonstration blocks, separated by blank lines, followed by the test equations and query. Each demonstration contains its equations and query on one line and its answer or calculation on the next. Answer-only output has the form `pen=6`. Calculation output restates equations, substitutes resolved values and reports each evaluated value in dependency order, ending with the queried variable. The test prompt ends immediately after its input line and a newline. Demonstrations reserve a name pool disjoint from test names; letter-name replication uses letter-name demonstrations. Task levels and name conditions are in Appendix B; additional equation and final-query formats are in Appendix I.

Code prompts. Both main code formats concatenate three ordinary-name worked examples, separated by blank lines, followed by the test function, its call and `Trace:`. There is no additional instruction or interpreter execution. Each example contains the program and input call, a `Trace:` line and an `Answer:` line. Table 3 gives the complete level-5 demonstration set used for Figure 1. Assemble one format by choosing its trace for each example. Both formats then append the same test program from that figure and the same `Trace:` prefix. Traces wrap here for typesetting; each is one line in the prompt. The additional formats are specified in Appendix M.

Parsing and denominators. For code few-shot output, parsing stops at the first blank line, so an invented subsequent problem cannot supply the answer. The final answer is the last `Answer: <integer>` in that block; answer-only continuations may instead start with the integer directly. A trace value is the last integer immediately following an equals sign in the target variable's first assignment clause, before `Answer:`. This retains the evaluated value in an expression-and-value trace rather than an operand. A comma followed by another assignment ends that clause. Interactive transcripts read the integer following the queried variable; comments read its end-of-line value. Free prose uses the final `Answer:` occurrence, or the last integer on the final nonempty line when that marker is absent.

For arithmetic few-shot output, the answer is the last standalone `<queried-name>=<integer>` clause before the first blank line. Free-text arithmetic uses a queried-name assignment or a final integer fallback. The reported main arithmetic name-error rate grades this final answer. Code intermediate correctness grades the first assignment against the program's executed value; final correctness grades the return value. Parse failures count as incorrect for accuracy and do not match a name-suggested digit. They remain in every behavioral denominator. Error-conditioned probes separately use the observed wrong-write cohort with verified generated prefixes.

Sampling and uncertainty. Main behavior uses demonstration seeds 7, 11 and 13, with three examples per set and greedy full-vocabulary decoding. The arithmetic panel contains thirteen checkpoints and 2,000 matched sets per task level. The code panel contains seven instruction-tuned checkpoints, with an additional 34B check, and 1,000 matched problems per target at levels 3 and 5. The central code cohort has 285 unique length-to-sum computations per set. Checkpoints and prompts differ between tasks; their results are not pooled.

Resampling retains both names and all demonstration repeats of a computation. Behavioral reliability requires a consistent sign in all three sets and a 95% interval excluding zero. The small-effect criterion uses a 90% interval within the prespecified two-point margin. Code timing and matched correctness intervals use 4,000 draws; the code equivalence audit uses 2,000. Intervals are pointwise, and probe intervals condition on the fixed independent layer selectors. These procedures are detailed in Appendices P, V and X; arithmetic repeated- computation checks are in Appendix H.

Probe fitting and records. Predictors use three initialization seeds, 0, 1 and 2, and full-batch SGD at learning rate 10^{-3} for 10,000 epochs. Training and evaluation are disjoint in computation, ignoring identifier spelling. Five-fold layer selection excludes the held-out computation. Cached prefix checks exclude the future answer token. Opti-

Program and input	Values-only trace	Expression-and-value trace

`def f(xs):`
`  u = min(xs)`
`  t = u + 1`
`  k = t * 2`
`  return k`
`f([0, 6, 5])`
	`u = 0, t = 1, k = 2`
	`Answer: 2`
		`u = min(xs) = 0, t = u + 1 = 1, k = t * 2 = 2`
		`Answer: 2`

`def f(xs):`
`  z = len(xs)`
`  r = z * 2`
`  k = r - 1`
`  return k`
`f([2, 4, 1])`
	`z = 3, r = 6, k = 5`
	`Answer: 5`
		`z = len(xs) = 3, r = z * 2 = 6, k = r - 1 = 5`
		`Answer: 5`

`def f(xs):`
`  p = len(xs)`
`  ww = p + 2`
`  u = ww // 2`
`  return u`
`f([3, 6, 0])`
	`p = 3, ww = 5, u = 2`
	`Answer: 2`
		`p = len(xs) = 3, ww = p + 2 = 5, u = ww // 2 = 2`
		`Answer: 2`

Table 3: Complete code demonstration set 7; prepend `Trace:` to each trace.

Name	Problem (answer: 6)
Neutral	`pen=1 + cup, cup=2 + 3; pen=?`
Congruent	`six=1 + cup, cup=2 + 3; six=?`
Incongruent	`four=1 + cup, cup=2 + 3; four=?`

Table 4: Matched names preserve the arithmetic. The misleading digit, 4, occurs nowhere else.

Level	Problem (neutral names)	Steps	Held	Distr.
1	`pen=1 + cup, cup=2; pen=?`	1	1	0
2	`pen=2 + 3, cup=1 + pen; cup=?`	2	0	0
3	`pen=1 + cup, cup=2 + 3; pen=?`	2	1	0
4	`pen=1 + cup, cup=2 + 3, box=4 + 5; pen=?`	2	1	1
5	`pen=1 + cup, cup=2 + box, box=1 + 2; pen=?`	3	2	0

Table 5: The five levels of Kudo et al. (2026): additions or subtractions needed (*Steps*), variables unresolved when first read (*Held*), and unused equations (*Distr.*). At level 1 the intermediate value is stated rather than computed.

mizer checks, controls, software versions and data records are in Appendices D.1, T and W.

B Arithmetic task and reproducibility

A problem contains single-digit variables, addition or subtraction, and a query. The five levels vary dependency order, depth and the presence of an unused equation (Table 5). Three same-level worked examples precede each test problem. The evaluated models are Gemma-3-12B-it, Gemma-3-4B, Gemma-3-4B-it, Llama-3.1-8B, Llama-3.1-8B-Instruct, Llama-3.2-3B, Llama-3.2-3B-Instruct, OLMo-2-1B-Instruct, OLMo-2-7B-Instruct, plus four derived checkpoints (Appendix C.3). Demonstrations use ordinary names for every word-name condition, with a reserved word pool disjoint from test names. Generation is greedy; only the test variable name changes between matched conditions.

We reimplement Kudo et al. (2026)'s task and probe recipe rather than using their code. The shared choices are single-digit outputs, same-level demonstrations, a 10-way linear residual-stream classifier, full-batch SGD at 10^{-3} for 10,000 epochs, and their span-by-layer-window patching design. For replication we train on letter names. The Llama-3.2-3B value-writing positions agree with their Table 3: equation 5 for the queried variable and 2 for the intermediate. Our additions are the conflicting names, matched baselines, demonstration-set claim rule and equivalence bound.

Pre-chain probe maxima are 30.4 and 47.9, compared with 17.8 and 33.2 in Kudo et al. (2026). The value-writing positions agree, while the accuracy difference may reflect training details.

B.1 Tokenizer checks

We test candidate names in five tokenizer contexts and retain words that form one token without merging with a neighbor. For Llama 3, all ten number words and 95 of 112 neutral candidates pass. Gemma and OLMo use the same lists; runtime checks confirm equal token lengths and po-

sitions within matched groups. Spaces around binary operators prevent merges such as `1-two`. No instance in the reported runs is excluded by the layout check.

Data, compute and software. Each level has 2,000 matched sets, each rendered in every condition, and 10,000 separately generated problems for training probes; every SGD probe is trained with 3 seeds. Every run here is inference and the probes are linear; the three adapter checkpoints of Appendix C.3 were trained in a separate project, and that compute is not counted below. Combined allocation time across the arithmetic and code experiments totals 222.5 GPU-hours on NVIDIA A30, H100 and H200 cards, an upper bound including idle and shared workers. The models are from the Llama 3 (Grattafiori et al., 2024), Gemma 3 (Gemma Team, 2025) and OLMo 2 (Team OLMo, 2024; Walsh et al., 2025) families, with the reasoning-tuned checkpoint from Bercovich et al. (2025), each used under its release licence for research. Models run in PyTorch (Paszke et al., 2019) with Transformers (Wolf et al., 2020), and the probes are trained in PyTorch; the L-BFGS refit of Appendix D.1 uses scikit-learn (Pedregosa et al., 2011), and the mixed-effects check uses statsmodels (Seabold and Perktold, 2010). Llama checkpoints use the Llama 3.1 or 3.2 Community License, Gemma uses the Gemma Terms of Use, OLMo uses Apache 2.0, and Nemotron uses the NVIDIA Open Model License. We use released weights for research inference and do not redistribute them. Versions are PyTorch 2.11.0 (CUDA 12.9), Transformers 5.14.1, NumPy 2.3.5, scikit-learn 1.9.1 and SciPy 1.18.0.

C Full arithmetic results and controls

C.1 Equivalence and accuracy floor

Following preregistration Amendment 3, equivalence requires a 90% cluster-bootstrap interval wholly inside the $\pm$2-point margin (two one-sided 5% tests). We resample matched sets with their demonstration-seed replicates together, using 4,000 bootstrap draws. Other behavioral intervals remain at 95%. For CoT accuracy, 61 of 88 facilitation and interference contrasts meet the same equivalence bound. Of the 15 reliable accuracy contrasts, 5 are also equivalent within the margin and 10 are not; a detectable effect can still be small. The written-value control uses the same matched pairs for its excess estimate and interval; the exception model's estimate pools all three demonstration sets and clusters on matched set.

C.2 Naming and written-value controls

Written values. We split generated chains by whether they state the target's true value. An observation enters the excess estimate only when its neutral twin falls in the same split. The estimate and its 95% interval use those same paired observations. Pooled over eligible cells, the excess is +0.08 points in 132 cells where the value is stated and -0.10 in 27 where it is not. The exception persists in the stated-value split (main text), so this split does not explain its effect.

Identifier-style names. At level 3, four models receive names such as `q4` instead of `four`, under three demonstration sets. Without CoT, queried-variable excess name errors range from -0.2 to +0.2 points; 0 of 4 cells are reliable. For number words on the same models, the range is -0.2 to +2.8, with 3 of 4 reliable. The numeric name-error effect is therefore stronger for number words in this comparison; congruent identifier names can still improve accuracy.

Word-class effects. With CoT at level 4, a number word on the queried variable costs Llama-3.2-3B 9.9 accuracy points whether the name agrees with the value or not. Those errors restate the wrong equation. This is distinct from answering the digit the name denotes, which is why the name-suggested value result cannot be read as removal of every naming effect.

Unused variables. A number word on the level 4 distractor changes accuracy by at most 1.2 points. Its no-CoT excess name errors range from +2.1 to +2.3; 2 of 2 cells are reliable. With CoT, the range is +0.0 to +1.0, with 0 of 2 reliable.

Mixed-effects check. We compare the paired bootstrap with a mixed logit model, `name_error ~ incongruent + C(seed)`, with a random intercept for matched set. Each twin's outcome refers to the same name-suggested digit. Both procedures use the same fixed subsample of 600 sets per cell. Of 57 fitted cells, 50 agree on whether the interval excludes zero; 11 additional cells have no outcome variation. The mixed fit supports an effect in 5 cells where the bootstrap does not and withdraws support in 2 cells where it does (Gemma-3-12B-it, value-only chain, queried variable; Llama-3.1-

Model	Regime	Neutral accuracy	number word on the *queried* variable		number word on the *intermediate* variable	
		(%)	helps	errors	helps	errors
Gemma-3-12B-it	CoT	100 [100,100]	+0.0	+0.0	+0.0	+0.0
Gemma-3-12B-it	direct	78 [76,81]	-3.2*	-0.2	-3.7*	+0.4
Gemma-3-4B-it	CoT	100 [100,100]	-0.0	+0.0	-0.0	+0.0
Gemma-3-4B-it	direct	55 [48,59]	+0.2	+0.9*	+0.6	+0.3
Llama-3.1-8B-Instruct	CoT	100 [99,100]	+0.2	+0.0	+0.3	+0.0
Llama-3.1-8B-Instruct	direct	52 [45,65]	+1.7	+0.1	+5.2*	+0.2
Llama-3.1-8B	CoT	95 [86,100]	-1.3	-0.1	+3.8	-0.0
Llama-3.1-8B	direct	53 [41,62]	+4.8*	+1.2*	+3.8*	-0.0
Llama-3.2-3B-Instruct	CoT	100 [100,100]	-0.0	+0.0	-0.2	+0.0
Llama-3.2-3B-Instruct	direct	44 [40,50]	-1.8*	+0.2	+0.5	+0.5
Llama-3.2-3B	CoT	98 [97,100]	-1.2	+0.1	+0.9	-0.2*
Llama-3.2-3B	direct	22 [13,31]	+5.4*	+2.8*	+1.1	+1.8
OLMo-2-7B-Instruct	CoT	95 [93,97]	+0.7*	-0.1	-2.7	-0.0
OLMo-2-7B-Instruct	direct	54 [50,58]	+0.6	-0.2	+3.8*	-0.4

Table 6: Arithmetic level 3. Accuracy (%); name effects (points). *Reliable across demonstration sets.

Model	Regime	Neutral accuracy	number word on the *queried* variable		number word on the *intermediate* variable	
		(%)	helps	errors	helps	errors
OLMo-2-1B-Instruct	CoT	47 [44,51]	-5.7*	+0.4	+7.0*	+3.2*
OLMo-2-1B-Instruct	direct	34 [30,38]	-0.0	+2.7*	+3.0*	-0.2

Table 7: The level-3 model below the preregistered 90% mean neutral-CoT accuracy floor, excluded from the accuracy-filtered behavioral table. Columns and claim markers follow Table 6.

Contrast	Cells	Equivalent	Reliable
CoT name errors	44	43	3
CoT accuracy	88	61	15
Direct name errors	44	31	13

Table 8: Equivalence within the two-point margin uses 90% intervals; reliability uses 95% intervals excluding zero and the same sign across all three demonstration sets. Each accuracy cell is one facilitation or interference contrast.

Nemotron-Nano-8B, CoT, intermediate variable). Subsampling also removes support from 1 starred Table 6 cell under both procedures (Gemma-3-4B-it, no CoT, queried variable). The mixed model uses variational Bayes, so its interval is an approximate posterior interval. We retain the full-data bootstrap as the primary estimate on the reported percentage-point scale.

Positional-copy control. Across 4110 generated CoT name errors over models, levels and demonstration sets, the final number in the comma-delimited segment before the answer is the name-suggested value in 13% of cases and the target's true value in 11%. A trailing name-suggested value is therefore absent from many errors. This check constrains the positional-copy explanation of Liu (2026), but does not identify what was copied in each case.

C.3 Derived checkpoints

We compare six checkpoints on the Llama-3.1-8B architecture: pretrained, instruction-tuned, single-task LoRA, multi-task LoRA, TIES-merged adapters and reasoning-tuned. Adapters are folded into the weights before evaluation. Shared tokenizers and identical matched problems permit paired comparisons (Table 10). A pretrained/instruct Gemma pair supplies a second family comparison.

Model	Level	Regime	Neutral accuracy	number word on the *queried* variable		number word on the *intermediate* variable	
			(%)	helps	errors	helps	errors
Llama-3.1-8B	1	CoT	99 [98,100]	+0.0	+0.0	+0.3	+0.0
Llama-3.1-8B	1	direct	74 [38,99]	+16.1*	+11.2*	+8.8*	+1.3
Llama-3.2-3B	1	CoT	97 [95,100]	+1.1	+0.1	-2.4	+0.3
Llama-3.2-3B	1	direct	46 [21,93]	+0.6	+5.9*	-3.9*	+2.6*
Llama-3.1-8B	2	CoT	100 [100,100]	-0.1	+0.0	-0.0	+0.0
Llama-3.1-8B	2	direct	36 [33,37]	+11.8*	+0.7	+2.6	+2.2
Llama-3.2-3B	2	CoT	94 [90,98]	+3.7	+0.5*	+1.0*	-0.1
Llama-3.2-3B	2	direct	29 [27,30]	+10.8*	+2.5*	+2.6*	+2.2
Llama-3.1-8B	4	CoT	88 [63,100]	+1.0	-0.0	-2.3	+0.2
Llama-3.1-8B	4	direct	28 [26,31]	+13.7*	+5.8	+6.0*	+0.2
Llama-3.2-3B	4	CoT	64 [26,85]	-10.2*	+0.7	-2.6*	+0.4
Llama-3.2-3B	4	direct	20 [16,23]	-1.6	+1.9	-0.2	+1.2*
Llama-3.1-8B	5	CoT	65 [43,100]	+1.2	-0.3	+0.1	-0.1
Llama-3.1-8B	5	direct	20 [13,26]	+20.2*	+20.5*	+0.9	+0.4
Llama-3.2-3B	5	CoT	69 [48,97]	-1.4	+0.1	-2.5	-0.1
Llama-3.2-3B	5	direct	12 [11,12]	+4.7*	+3.5*	+1.1*	+0.5

Table 9: Additional task levels for the two Llama models. Columns and claim markers follow Table 6; level 1 states the intermediate value directly.

Task accuracy complicates interpretation. The multi-task model's reliable negative excess (-0.58) accompanies lower neutral accuracy (46.1%, versus 52.2%). The reasoning-tuned checkpoint has the lowest accuracy (39.4%). It also requires a system instruction to enable long-form reasoning; our raw-completion prompts leave that mode off. The result concerns the checkpoint under this prompt, not reasoning mode in deployment. Across these checkpoints, CoT excess name errors are at most 0.20 points in magnitude.

C.4 Values without equations

Our chain restates each equation before substituting into it, as Kudo et al.'s does. Their Tables 5–6 also report a format that writes only the subresults, `cup=5, pen=6`. We test this format at level 3 with three demonstration sets. Two predictions were registered for both Llama models: that the excess name errors at the intermediate's value step would exceed the 2-point margin, and that the value would still be decodable there (probe accuracy at least 0.85), making any failure one of readout.

Both predictions fail as registered. Llama-3.1-8B writes the name-suggested value at the intermediate's value step +0.15 points above its twin, inside the margin. Llama-3.2-3B is at +9.20, which passes our claim rule, but across the three demonstration sets that is +1.7 / +24.2 / +1.7, so one set carries most of it. More informative is what the probes say. At the step that writes the value, the true value is decodable at 0.81 and 0.59 for Llama-3.1-8B and at 0.61 and 0.79 for Llama-3.2-3B, against 1.00 and 0.99 for the same model under the full chain. Neutral accuracy falls with it, to 54% and 31%.

Removing the equations lowers both accuracy and true-value probe readouts. It therefore fails to isolate a readout error while preserving those measures.

D Probe and patching controls

Probes predict the true digit, the name-suggested digit and their probability margin. The name-identity control assigns a fixed random digit to each name; selectivity subtracts its accuracy from true-value accuracy. Layers in the supplied-calculation analysis are selected by neutral selectivity, averaged over three probe seeds. P1 is the

Tuning	Model	Queried	Intermediate	Acc.
base	Llama-3.1-8B	+1.20*	-0.02	53.4
instruct	+ instruct	+0.15	+0.22	52.2
single-task finetune	+ task LoRA	+0.05	+0.23	52.2
multi-task finetune	+ multi-task LoRA	-0.58*	-0.08	46.1
weight merge	+ TIES merge	-0.10	+0.10	51.9
reasoning-tuned	Nemotron-Nano-8B	+1.35*	+0.33	39.4
base	Gemma-3-4B	+0.08	+0.02	60.2
instruct	+ instruct	+0.88*	+0.33	54.9

Table 10: Level 3, no CoT, name errors above the matched twin's rate, mean over three demonstration sets. *Acc.* is neutral accuracy. * as in Table 6. The bottom block repeats the pretrained-to-instruct step in a second family, Gemma-3-4B.

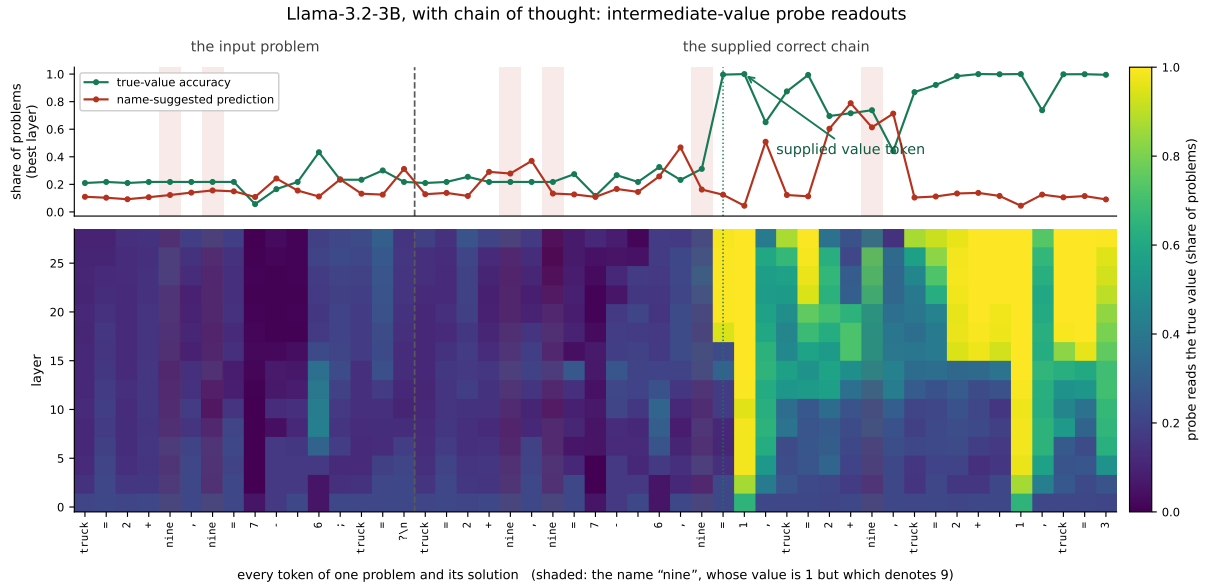


Figure 5: Llama-3.2-3B intermediate-value readouts under supplied correct calculations. Rates pool problems; tokens illustrate one example.

name token, P2 the end of its defining equation, P3 the query, P4 the position before its value is written, and P5 the position before the final answer. P1 is a reference, and P3 can reveal name identity rather than a computed value. Figure 5 gives the token and layer sweep supporting the main readout analysis.

Patches replace residual-stream activations at matched name-token positions, at individual layers or across all layers. The neutral-twin source tests removal of the name error; a different ordinary name tests damage to correct answers; an alternative misleading name tests specificity. The span sweep instead replaces an equation, the query or the value-writing segment in four-layer windows. Figure 6 supports the early-layer localization. Table 11 gives the answer-only control rates and denominators for both Llama models. OLMo's supplied-calculation controls are reported in Table 13. We also reverse the main contrast, transplanting misleading-name states into the ordinary-name twin. OLMo's largest injection rate across layers and both variables is 2.7% at layer 5; word-control damage there is 0.4%. The injection itself damages 7.0% of correct answers. This selected maximum is descriptive.

Instance-level association. Within each model, level, regime, variable and position, we regress name errors on the probe margin, $\log p_{\text{true}} - \log p_{\text{name}}$, where p_{name} is the probe probability of the name-suggested value, with standard errors clustered on matched set. The layer is selected by neutral accuracy. Of 63 fits, 29 have $p < .05$: 24 coefficients are negative and 5 positive. The direction is not consistent, and 4 positive results come from the model supplying most fits. This leaves the instance-level association unresolved.

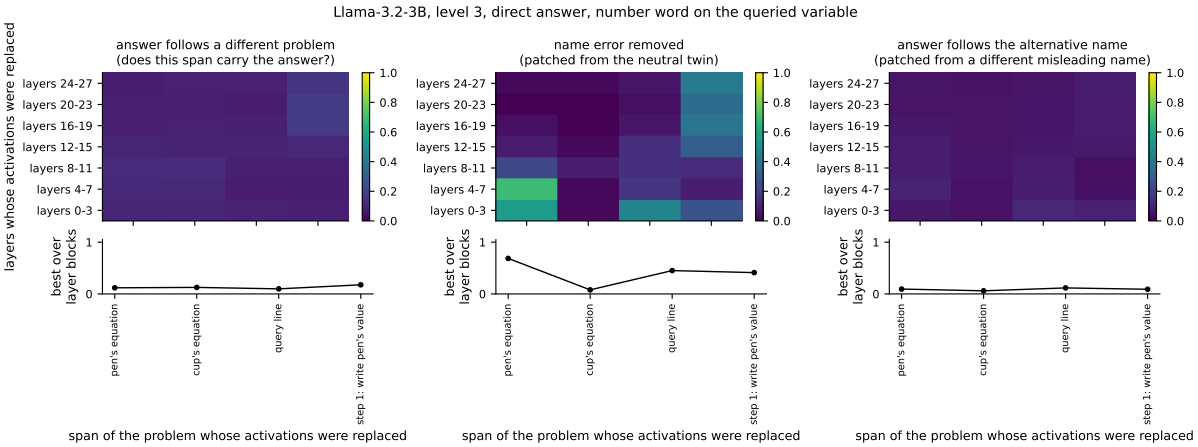


Figure 6: Llama-3.2-3B answer-only patches in four-layer windows. Bottom: maximum over windows.

Model	Variable	Pairs	Name errors	Correct	Neutral	Alternative	Word damage
Llama-3.2-3B	queried	2000	172	272	51.7	34.9	22.4
Llama-3.1-8B	intermediate	2000	72	838	30.6	30.6	8.5

Table 11: All-layer answer-only patches, demonstration set 7. Neutral and alternative: percentage of baseline name errors removed. Word damage: percentage of correct ordinary-name answers changed; *Correct* gives that denominator.

The model-level pattern is also sensitive to panel composition (Appendix E).

D.1 Probe recipe and patching scope

We check two implementation choices inherited from Kudo et al. (2026): the probe optimizer and the name occurrences included in a patch.

Recipe. We refit the level-3 probes for Llama-3.2-3B with L-BFGS, and with SGD on standardized inputs, against the reported recipe.

All three recipes give the same qualitative result: the value is decodable where the chain writes it (0.96–1.00) and not at the end of its defining equation (0.18–0.50). The query position moves, from 0.271 under the reported recipe to 0.393 under standardization, within a range of 0.18–0.39 over recipes and variables. We already report that position as the presence of the name rather than a decoded value, because its control task scores 0.97, so this variation does not change the interpretation of that position.

Scope. Patching every occurrence of the name includes the copy the model writes in its own output. In the direct regime, restricting the patch to the name's occurrences in the prompt leaves the intermediate variable essentially unchanged, since its name never appears in the answer: removal is 43% at layers 0–3 under both scopes. For the queried variable the best windows remove similar shares (52% at layers 4–7 prompt-only, against 53% at layers 0–3), the prompt-only patch removes less in the later windows, and the two disagree most in the first window, where the prompt-only patch removes 3% and damages 76% of correct answers. That is mechanical: the queried variable's name also sits at the position we read, so patching its prompt copies alone leaves the model reading a name whose early representation no longer matches. The localization we report does not depend on the choice.

E Task competence and value readouts

Table 14 compares both variables for 9 models. Layers are selected by neutral selectivity, averaging three probe seeds. Task accuracy and paired excess name errors pool demonstration sets 7, 11 and 13; probes use set 7. The within-model difference holds overall task accuracy fixed but does not isolate a causal effect of the readout. We average the two variable readouts within each model before computing descriptive correlations. Table 15 repeats them after omitting each model. Roles are not independent observations. With this small, heterogeneous panel, correlations cannot distinguish overall competence from the role of decodable in-

Position	Variable	SGD (reported)	L-BFGS	standardized
defining equation	queried	0.184	0.190	0.183
defining equation	intermediate	0.496	0.422	0.420
query	queried	0.183	0.316	0.280
query	intermediate	0.271	0.366	0.393
value step	queried	1.000	1.000	1.000
value step	intermediate	0.982	0.962	0.972
answer	queried	1.000	1.000	1.000

Table 12: Best-layer true-value accuracy on neutral problems under three probe recipes (proportions from 0 to 1).

Target	Read	Errors	Neutral removed	Word dmg.	Alt. removed	Alt. followed
v1	value	0	–	1.6	–	0.0
v1	answer	0	–	1.6	–	0.0
v2	value	28	92.9	7.3	82.1	4.0
v2	answer	3	100.0	0.8	100.0	0.6

Table 13: OLMo-2-1B-Instruct, supplied correct calculations, 500 matched pairs per target. All-layer rates (%): neutral/alternative name-error removal, word-control damage to correct predictions, and predictions following the alternative digit. Roles v1/v2: queried/intermediate; –: undefined.

formation.

F Gold-trained transfer to generated chains

We read the state before the first numeric assignment to the target in each saved generated chain. Expression operands are not counted as completed value writes; tokens that include any part of the supplied value are excluded. A fixed sample of 200 matched sets is selected by a hash independent of outcomes, shared across models, variables and demonstration sets. All additional chains with name errors and their neutral twins are evaluated separately. Neutral gold-chain probes use 10,000 separate training problems, three probe seeds and the original neutral-selectivity layer. Table 16 tests transfer to held-out neutral generated chains. A high-accuracy interpretation requires at least 90% neutral calibration in that demonstration set. Cases below that threshold yield descriptive readouts only. The augmentation is selected by observed errors and cannot estimate their population rate. Appendix G separately trains probes on generated neutral chains and tests the same held-out prefixes. Llama-3.2-3B meets the neutral transfer threshold in 6 of 6 cells, with calibration accuracy from 97.2 to 100.0%. OLMo-2-1B meets the neutral transfer threshold in 0 of 6 cells, with calibration accuracy from 19.4 to 62.2%. Name-error cases in the competent model are few; error-conditioned readouts cannot establish a general mechanism from that subset. Failed neutral calibration prevents a strong generated-chain interpretation for the exception model.

G Training probes on generated neutral chains

Gold-trained probe transfer (Appendix F) may fail because the states along generated chains differ from those along the supplied correct chain. We therefore train on freely generated neutral chains from 2,000 source problems and select layers on 400 separate validation problems. Canonical equations partition the source pool, keeping renamed versions of a computation together. The training and validation pools contain 630 and 155 computations; both exclude all test and demonstration computations. These counts distinguish independent computations from repeated name renderings. Generation is greedy with demonstration set 7 and a 128-token limit. The probe sees the hidden state before the target variable's first numeric assignment, with every token boundary checked to exclude the value digit. Missing or unsafe boundaries are excluded and counted. We retain all usable neutral chains regardless of whether the model writes the correct value, using the executed true value as the training label. Three probe seeds use the original full-batch SGD recipe (learning rate 0.001, 10,000 epochs). States are stored in float32 and nonfinite states or fitted weights are rejected. Layers are searched at stride

Model	Role	Layer	Task	Neutral	Name ID	Misleading	Excess
Gemma-3-12B-I	v1	40	100.0	100.0	13.1	100.0	0.0
Gemma-3-12B-I	v2	42	100.0	100.0	11.9	100.0	0.0
Gemma-3-4B	v1	25	100.0	100.0	10.0	100.0	0.0
Gemma-3-4B	v2	28	100.0	99.8	12.4	100.0	0.0
Gemma-3-4B-I	v1	29	100.0	100.0	7.4	99.9	0.0
Gemma-3-4B-I	v2	28	100.0	99.8	10.0	99.9	0.0
Llama-3.1-8B	v1	20	95.2	100.0	20.5	100.0	-0.1
Llama-3.1-8B	v2	22	95.2	99.4	24.9	99.2	-0.0
Llama-3.1-8B-I	v1	20	99.7	100.0	21.0	100.0	0.0
Llama-3.1-8B-I	v2	20	99.7	100.0	25.7	100.0	0.0
Llama-3.2-3B	v1	19	98.3	99.9	22.7	100.0	0.1
Llama-3.2-3B	v2	21	98.3	97.6	31.5	99.6	-0.2
Llama-3.2-3B-I	v1	18	100.0	99.9	26.3	100.0	0.0
Llama-3.2-3B-I	v2	19	100.0	99.9	29.5	99.9	0.0
OLMo-2-1B	v1	14	47.2	99.7	48.9	100.0	0.4
OLMo-2-1B	v2	0	47.2	21.8	15.0	21.8	3.2
OLMo-2-7B	v1	22	95.3	100.0	37.7	100.0	-0.1
OLMo-2-7B	v2	26	95.3	99.4	47.5	97.6	-0.0

Table 14: Task and true-digit probe accuracies (%), name-identity control accuracy (%), and excess name errors (points). Roles v1/v2 are queried/intermediate. Selectivity is neutral minus name-ID accuracy. Below-floor models remain descriptive.

Omitted model	Task–excess	Probe–excess	Task–probe
All models	-0.99	-1.00	+1.00
Gemma-3-12B-I	-0.99	-1.00	+1.00
Gemma-3-4B	-0.99	-1.00	+1.00
Gemma-3-4B-I	-0.99	-1.00	+1.00
Llama-3.1-8B	-0.99	-1.00	+1.00
Llama-3.1-8B-I	-0.99	-1.00	+1.00
Llama-3.2-3B	-0.99	-1.00	+1.00
Llama-3.2-3B-I	-0.99	-1.00	+1.00
OLMo-2-1B	+0.98	+0.78	+0.85
OLMo-2-7B	-0.99	-1.00	+1.00

Table 15: Descriptive Pearson correlations of model means; Spearman values and within-model variable differences are in the saved summary. No significance or causal claim is based on these coefficients.

two, including the final layer, and chosen only by mean validation accuracy with the lowest-layer tie break. The name-identity control is trained separately and does not select layers. Table 17 reports the usable source counts and selected layers. Evaluation uses exactly the earlier fixed 200-set sample and separately reported additional name-error cases under demonstration sets 7, 11 and 13. Table 18 gives held-out neutral calibration and boundary coverage. Intervals use 4,000 canonical-computation bootstrap draws, keeping repeated names together. The 90% calibration threshold applies to the point estimate, not the lower interval endpoint. Llama passes all six calibration cells; OLMo passes none. Table 19 retains error readouts with their denominators. OLMo's readouts remain descriptive. Llama has only one name-error case in the primary sample and thirteen additional observed writes; these rare, selected cases cannot establish a general mechanism.

H Paired change from answer-only to written calculations

We compare prompting conditions on exactly the same matched sets and demonstration seeds used in the 44 name-error equivalence cells. Each observation subtracts the ordinary-name rate from the misleading-name rate, then subtracts the answer-only contrast from the CoT contrast. Negative changes indicate fewer added name-suggested answers. We bootstrap matched sets, keeping all three demonstration repeats together (4,000 draws). Reliable reductions require a negative change in every demonstration set and a 95% interval below zero. The panel is descriptive across related checkpoints; its cells are not independent model replications. Of the 44 paired changes, 10 meet that rule; 19 have intervals below zero.

Model	Set	Role	Neutral coverage	Calibration (%)	Sample errors	Extra errors
OLMo-2-1B	7	v1	194/200	62.2 [55.2, 68.7]	8	69
OLMo-2-1B	7	v2	186/200	19.4 [14.0, 25.3]	11	82
OLMo-2-1B	11	v1	187/200	55.3 [48.3, 62.6]	7	59
OLMo-2-1B	11	v2	129/200	21.7 [14.7, 28.7]	2	21
OLMo-2-1B	13	v1	193/200	52.7 [45.6, 59.8]	17	159
OLMo-2-1B	13	v2	124/200	24.2 [16.9, 32.3]	1	30
Llama-3.2-3B	7	v1	200/200	98.0 [96.0, 99.5]	1	8
Llama-3.2-3B	7	v2	200/200	97.2 [94.8, 99.0]	0	0
Llama-3.2-3B	11	v1	199/200	100.0 [100.0, 100.0]	0	0
Llama-3.2-3B	11	v2	200/200	100.0 [100.0, 100.0]	0	0
Llama-3.2-3B	13	v1	199/200	99.5 [98.5, 100.0]	0	5
Llama-3.2-3B	13	v2	200/200	99.5 [98.5, 100.0]	0	0

Table 16: Gold-trained probe accuracy on neutral generated chains [95% interval], and eligible name-error counts. Extra cases exclude the fixed sample. Roles v1/v2 are queried/intermediate. Calibration is required before interpreting error readouts.

Model	Role	Train n	Validation n	Layer	Validation (%)	Name ID (%)
OLMo-2-1B-I	v1	1924	390	14	47.9	41.6
OLMo-2-1B-I	v2	1877	376	16	87.8	86.6
Llama-3.2-3B	v1	1994	398	22	97.5	26.8
Llama-3.2-3B	v2	2000	400	20	99.2	28.6

Table 17: Usable generated-boundary counts and validation-selected layers. Roles v1/v2 are queried/intermediate. Source cohorts are computation-disjoint; validation scores are not test estimates.

A sensitivity bootstrap that also clusters repeated arithmetic computations yields 10 reliable reductions. Intervals are pointwise; the count is not a multiplicity-corrected family-wide test. Before writing calculations, 31 cells already meet the two-point equivalence bound, compared with 43 afterward. Thus the equivalence count is not a count of significant improvements. Accuracy gains accompany many reductions and do not identify an effect independent of arithmetic competence.

I Response formats and fixed correct calculations

These exploratory controls use Llama-3.2-3B, Llama-3.1-8B and OLMo-2-1B-Instruct, level 3 and demonstration seeds 7, 11 and 13. The protocol and cohorts were frozen before collecting outputs. Intervals use 4,000 bootstrap draws over arithmetic computations, keeping repeated names and demonstrations together. Intervals are pointwise, without multiplicity correction; zero-event bootstrap intervals do not establish zero population rates.

Equations and repeated names. We cross numeric equations with name repetition in demonstrated outputs. For example, named equations write `cup=2+3=5, pen=1+5=6`; unnamed equations write `2+3=5, 1+5=6`; values-only formats write `cup=5, pen=6` or `5, 6`. All end with the same `Answer: 6`. Only examples provide correct calculations; test outputs are freely generated. Repeated ordinary-word notes before each example output match full prompt token counts across formats and matched names. Generated lengths are measured rather than forced to match. We select 200 independent test computations, each rendered with ordinary names and with a misleading name on either variable, yielding 600 paired observations per target across demonstrations. Calibration uses 100 independent ordinary-name computations, disjoint from tests and demonstrations. We try 3, 8 and 16 examples, seeking at least 90% accuracy in every format and a range within two points. No model passes; all use 16 examples. A held-out comparison matches competence only if both ordinary-name accuracies reach 90% and their paired 90% difference interval fits within ± 2 points. Only the named-versus-unnamed equation comparisons in the two Llama models pass (four target comparisons). None of the equation-versus-values comparisons passes.

Answering after a correct calculation. We resume after actual original generations for which every restatement, substitution and value matches the executable gold calculation. Both naming twins must qualify. Within each naming condition,

Model	Role	Set	Neutral coverage	Calibration (%)	Name ID (%)	Pass
OLMo-2-1B-I	v1	7	194/200	61.3 [54.1, 68.6]	37.3 [31.2, 43.4]	no
OLMo-2-1B-I	v1	11	187/200	52.8 [45.1, 60.3]	32.4 [26.6, 38.5]	no
OLMo-2-1B-I	v1	13	193/200	50.3 [43.0, 57.4]	35.2 [29.1, 41.6]	no
OLMo-2-1B-I	v2	7	186/200	78.0 [71.5, 83.9]	81.9 [76.9, 86.7]	no
OLMo-2-1B-I	v2	11	129/200	80.1 [72.8, 86.8]	56.6 [49.6, 63.3]	no
OLMo-2-1B-I	v2	13	124/200	71.8 [64.2, 78.4]	46.8 [37.8, 55.6]	no
Llama-3.2-3B	v1	7	200/200	98.5 [96.7, 100.0]	18.5 [13.8, 23.7]	yes
Llama-3.2-3B	v1	11	199/200	100.0 [100.0, 100.0]	13.9 [9.5, 18.5]	yes
Llama-3.2-3B	v1	13	199/200	97.2 [94.2, 99.5]	16.9 [12.3, 21.8]	yes
Llama-3.2-3B	v2	7	200/200	96.3 [93.6, 98.6]	28.8 [22.7, 35.0]	yes
Llama-3.2-3B	v2	11	200/200	100.0 [100.0, 100.0]	17.2 [12.4, 22.1]	yes
Llama-3.2-3B	v2	13	200/200	97.7 [95.5, 99.3]	17.8 [13.3, 22.8]	yes

Table 18: Held-out primary neutral generated chains. Coverage counts valid pre-value boundaries over all primary neutral rows. Accuracy intervals are 95% canonical-computation bootstrap intervals; pass means the calibration point estimate meets 90%.

Model	Role	Set	Sample n	Sample accuracy	Extra n	Extra accuracy
OLMo-2-1B-I	v1	7	8	4.2 [0.0, 12.5]	69	2.9 [0.0, 7.0]
OLMo-2-1B-I	v1	11	7	19.0 [0.0, 38.1]	59	10.7 [4.2, 18.8]
OLMo-2-1B-I	v1	13	17	21.6 [7.8, 39.2]	159	13.6 [8.6, 19.4]
OLMo-2-1B-I	v2	7	11	33.3 [15.2, 51.5]	82	18.7 [11.4, 27.2]
OLMo-2-1B-I	v2	11	2	0.0 [0.0, 0.0]	21	17.5 [3.3, 31.8]
OLMo-2-1B-I	v2	13	1	0.0 [0.0, 0.0]	30	25.6 [13.7, 39.1]
Llama-3.2-3B	v1	7	1	0.0 [0.0, 0.0]	8	12.5 [0.0, 42.9]
Llama-3.2-3B	v1	11	0	–	0	–
Llama-3.2-3B	v1	13	0	–	5	0.0 [0.0, 0.0]
Llama-3.2-3B	v2	7	0	–	0	–
Llama-3.2-3B	v2	11	0	–	0	–
Llama-3.2-3B	v2	13	0	–	0	–

Table 19: True-digit accuracy (%) [95% interval] on actual name errors, with the fixed sample and additional error-selected cases separated. Extra cases exclude the primary sample. Dashes denote empty strata. OLMo fails neutral calibration, so its readouts are descriptive.

the input and complete generated calculation are fixed. We cross a final query repeating the queried name with `the original requested value=?`, and an output starting with `Answer:` with one starting with `Answer: pen=`. The instruction always requests the original problem's value. These continuations are conditional on successful original calculations; they do not estimate an effect of making a calculation correct. Tail lengths and output markers differ by design. Each Llama model supplies 200 pairs per demonstration set and target. OLMo supplies 144/108/116 queried-target pairs and 44/8/25 intermediate-target pairs. Across demonstration sets, the queried-target cohorts contain 210/234/205 distinct computations for Llama-3B/Llama-8B/OLMo; intermediate-target cohorts contain 218/233/53. Generation budgets are 128 tokens for format tests and 16 for final continuations, with greedy full-vocabulary decoding. Unparsed outputs count as incorrect. For OLMo, repeating the queried name lowers added errors by 12.2 points [8.4, 16.1] in digit outputs, but ordinary-name accuracy rises from 78.0% to 96.2%. An assignment output also changes accuracy. No OLMo final-format comparison meets the competence criterion. With the misleading name on the intermediate variable, all added-error estimates are zero; OLMo has one suggested-digit answer in each naming condition in the repeated-query/digit format. The query repeats an ordinary name in this control. Neither Llama produces a suggested-digit answer in any final format. These observations distinguish correct calculations from reliable final answering, without identifying a single responsible feature.

J Code task levels and identifiers

Levels vary dependency depth, distractors and which assignment receives the misleading name. The primary comparison uses level 5. A function is generated once; renaming preserves its statements, input and executable result. Table 25 lists

Model	Level/role	Answer-only	CoT	Paired change [95% interval]	Accuracy gain
Llama-3.2-3B	L1/v1	+5.90	+0.12	-5.78 [-6.55, -5.02]*	+50.7
Llama-3.2-3B	L1/v2	+2.62	+0.30	-2.32 [-2.87, -1.77]*	+50.7
Llama-3.1-8B	L1/v1	+11.20	+0.00	-11.20 [-12.05, -10.37]*	+25.6
Llama-3.1-8B	L1/v2	+1.33	+0.02	-1.32 [-1.68, -0.97]	+25.6
Llama-3.2-3B	L2/v1	+2.50	+0.53	-1.97 [-2.65, -1.28]*	+65.1
Llama-3.2-3B	L2/v2	+2.20	-0.05	-2.25 [-2.92, -1.63]	+65.1
Llama-3.1-8B	L2/v1	+0.73	+0.02	-0.72 [-1.25, -0.20]	+64.4
Llama-3.1-8B	L2/v2	+2.20	+0.00	-2.20 [-2.73, -1.63]	+64.4
Llama-3.2-3B	L3/v1	+2.78	+0.13	-2.65 [-3.30, -2.00]*	+76.1
Llama-3.2-3B	L3/v2	+1.78	-0.17	-1.95 [-2.42, -1.48]	+76.1
Llama-3.2-3B-I	L3/v1	+0.17	+0.00	-0.17 [-0.62, +0.28]	+56.0
Llama-3.2-3B-I	L3/v2	+0.47	+0.00	-0.47 [-1.05, +0.12]	+56.0
Llama-3.1-8B	L3/v1	+1.20	-0.13	-1.33 [-1.82, -0.85]*	+41.8
Llama-3.1-8B	L3/v2	-0.02	-0.02	+0.00 [-0.47, +0.47]	+41.8
Llama-3.1-8B-I	L3/v1	+0.15	+0.00	-0.15 [-0.60, +0.28]	+47.5
Llama-3.1-8B-I	L3/v2	+0.22	+0.02	-0.20 [-0.68, +0.28]	+47.5
Gemma-3-4B-I	L3/v1	+0.88	+0.00	-0.88 [-1.38, -0.38]*	+45.1
Gemma-3-4B-I	L3/v2	+0.33	+0.02	-0.32 [-0.80, +0.15]	+45.1
Gemma-3-12B-I	L3/v1	-0.23	+0.00	+0.23 [-0.08, +0.55]	+22.4
Gemma-3-12B-I	L3/v2	+0.40	+0.00	-0.40 [-0.83, +0.02]	+22.4
OLMo-2-1B-I	L3/v1	+2.73	+0.45	-2.28 [-3.28, -1.30]*	+13.5
OLMo-2-1B-I	L3/v2	-0.20	+3.17	+3.37 [+2.37, +4.37]	+13.5

Table 20: Excess name errors and paired CoT-minus-answer-only changes (percentage points). Accuracy gain is the ordinary-name accuracy change. Roles v1/v2/v3 are the first/second/third variables in the problem; the queried role is v2 at level 2 and v1 otherwise. *Reliable reduction across all demonstration sets.

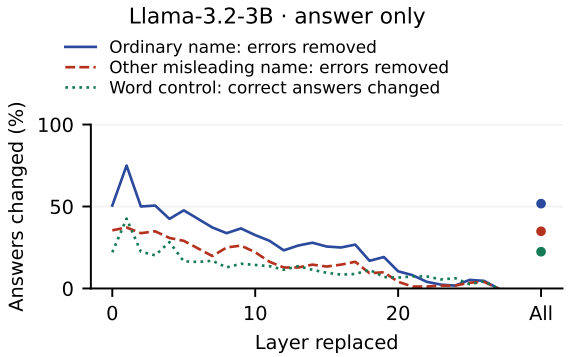


Figure 7: Arithmetic name-site patches, Llama-3.2-3B, queried variable, set 7. All replaces every layer.

the operation suggested by each identifier. Sum and length are admissible name-suggested operations on the aggregate step; minimum and maximum are excluded because their result is already present in the input.

K Identifier meanings

For each identifier and model, we compare log-probabilities of candidate right-hand sides after a prefix such as `def f(xs):\n total =`. The highest-scoring operation records what that identifier evokes. Of 25 names, 12 match the intended operation in at least 80% of models. Most sum-family names instead evoke length in 2 of seven models. This check matters when interpreting a small effect: an identifier need not suggest the same operation to every model. The design planned to retain only names passing the 80% panel gate. The runs instead retained the full name table, so the reported comparisons depart from that selection rule. The results concern these named groups; they do not establish that every model reads each name as intended.

L Operation and name pairings

Table 27 reports all tested aggregate/name pairings. The reverse of the main length-to-sum conflict is weak at level 5: sum-computed variables with length-like names have excess within two points for models above the neutral-accuracy floor. At level 3, OLMo-2-7B has +4.4 points of excess in that pairing. Gemma-3-4B shows no main-cell effect, consistent with its identifier-meaning check. OLMo-2-1B's neutral accuracy is only 37.8%, making its small, broadly distributed effects harder to interpret as a specific trace failure.

M Demonstration formats

All few-shot comparisons hold programs, demonstration identities and model weights fixed. The values-only format writes `qq = 3`; the expression-and-value format writes `qq = len(xs) = 3`; the interactive Python format writes `>>> qq` followed by

Model	Level/role	Answer-only	CoT	Paired change [95% interval]	Accuracy gain
OLMo-2-7B-I	L3/v1	-0.23	-0.08	+0.15 [-0.33, +0.60]	+41.0
OLMo-2-7B-I	L3/v2	-0.42	-0.03	+0.38 [-0.13, +0.87]	+41.0
Llama-8B single-task	L3/v1	+0.05	+0.00	-0.05 [-0.50, +0.42]	+47.6
Llama-8B single-task	L3/v2	+0.23	-0.02	-0.25 [-0.75, +0.23]	+47.6
Llama-8B multi-task	L3/v1	-0.58	+0.00	+0.58 [+0.23, +0.95]	+53.7
Llama-8B multi-task	L3/v2	-0.08	+0.00	+0.08 [-0.38, +0.53]	+53.7
Llama-8B merged	L3/v1	-0.10	+0.00	+0.10 [-0.30, +0.50]	+47.8
Llama-8B merged	L3/v2	+0.10	+0.00	-0.10 [-0.58, +0.38]	+47.8
Nemotron-Nano-8B	L3/v1	+1.35	+0.08	-1.27 [-1.78, -0.77]	+58.8
Nemotron-Nano-8B	L3/v2	+0.33	+0.20	-0.13 [-0.63, +0.38]	+58.8
Gemma-3-4B	L3/v1	+0.08	+0.00	-0.08 [-0.53, +0.35]	+39.8
Gemma-3-4B	L3/v2	+0.02	+0.00	-0.02 [-0.52, +0.48]	+39.8
Llama-3.2-3B	L4/v1	+1.93	+0.68	-1.25 [-1.78, -0.70]	+44.4
Llama-3.2-3B	L4/v2	+1.17	+0.37	-0.80 [-1.32, -0.27]	+44.4
Llama-3.1-8B	L4/v1	+5.83	-0.03	-5.87 [-6.62, -5.17]	+59.5
Llama-3.1-8B	L4/v2	+0.23	+0.20	-0.03 [-0.45, +0.37]	+59.5
Llama-3.2-3B	L5/v1	+3.45	+0.08	-3.37 [-4.10, -2.67]*	+57.5
Llama-3.2-3B	L5/v2	+0.52	-0.07	-0.58 [-1.17, +0.03]	+57.5
Llama-3.2-3B	L5/v3	-0.35	-0.08	+0.27 [-0.35, +0.88]	+57.5
Llama-3.1-8B	L5/v1	+20.50	-0.30	-20.80 [-21.85, -19.75]*	+45.1
Llama-3.1-8B	L5/v2	+0.45	-0.15	-0.60 [-1.28, +0.05]	+45.1
Llama-3.1-8B	L5/v3	+0.33	+0.08	-0.25 [-0.92, +0.42]	+45.1

Table 21: Excess name errors and paired CoT-minus-answer-only changes (percentage points). Accuracy gain is the ordinary-name accuracy change. Roles v1/v2/v3 are the first/second/third variables in the problem; the queried role is v2 at level 2 and v1 otherwise. *Reliable reduction across all demonstration sets.

3; comment mode annotates each assignment with its value. These are model-generated transcripts, not interpreter executions. Prose asks for step-by-step reasoning without demonstrations. Table 29 gives the paired excess and interval for each few-shot format.

On the main conflict, interactive-transcript excess is +9.5 and +12.6 points for OLMo-2-7B and Llama-3.2-3B, compared with +0.2 for Llama-3.1-8B. Comments leave +6.8 points in Llama-3.2-3B and near-zero excess in the other two. Prose has little initial name errors, so asking it to state or recompute values does not test removal of the large trace effect. The prediction that interactive transcripts would stay within a factor of two of trace was not supported; the comment prediction held only for two models.

The paired trace-minus-trace-expr name-error-rate reductions are 55 points [49.9, 60.2], 43 [38.2, 47.7] and 12 [8.8, 16.0] for OLMo-2-7B, Llama-3.2-3B and Llama-3.1-8B. These 95% intervals keep each matched program's demonstration-seed repeats together.

N Probe and patching details

We follow Kudo et al.'s 10-class linear-probe recipe, with three seeds and neutral-only training. Each probe predicts a target digit from one residual-stream position and layer. Name-identity controls use deterministic random labels keyed on the identifier (Hewitt and Liang, 2019). Neutral names and values vary independently, and misleading names are absent from training. The original implementation reused test programs in training. The reported reruns replace those scores: they train on 2,000 independent neutral programs and evaluate 285 misleading-name programs, with overlap guards that ignore name spelling. Table 30 reports the name controls alongside digit accuracy.

Figure 9's probe curves use demonstration set 7. Repeating training and evaluation with sets 11 and 13 gives true-digit accuracies of 95–97%, 94–100% and 100–100% across the three sets. Each repeat uses the same independent training programs and three probe seeds; its layer is selected by that set's neutral accuracy. These repeats assess prompt sensitivity, not a causal role for the decoded digit. Table 30 gives both conditions' readouts and controls.

We supply the correct trace token by token when caching probe inputs. The decision position is the = before the first traced value; later values cannot affect this position under causal attention. Patching uses only the prompt and this trace prefix. Name patches replace every aligned occurrence of the identifier; decision patches replace only that writing position. Both use one layer at a time and include a neutral-for-neutral

Model	Example output	Correct answers (%) Ordinary	Queried	Intermediate	Added name errors [95% interval] Queried	Intermediate
Llama-3.2-3B	Equations + names	100.0	100.0	100.0	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]
Llama-3.2-3B	Equations only	99.0	99.0	99.5	+0.0 [+0.0, +0.0]	-0.2 [-0.5, +0.0]
Llama-3.2-3B	Values + names	66.3	67.0	68.5	+0.0 [-1.3, +1.3]	+0.5 [-1.0, +2.0]
Llama-3.2-3B	Values only	22.3	20.8	21.5	+0.8 [-1.2, +2.8]	+1.2 [-0.8, +3.2]
Llama-3.1-8B	Equations + names	100.0	100.0	100.0	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]
Llama-3.1-8B	Equations only	100.0	100.0	100.0	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]
Llama-3.1-8B	Values + names	84.5	82.5	86.8	-0.3 [-1.3, +0.5]	-0.8 [-2.0, +0.0]
Llama-3.1-8B	Values only	38.0	36.5	36.0	-0.5 [-2.2, +1.0]	+0.2 [-1.2, +1.5]
OLMo-2-1B-I	Equations + names	99.7	94.7	93.5	+0.2 [+0.0, +0.5]	+0.3 [+0.0, +0.8]
OLMo-2-1B-I	Equations only	67.3	44.3	53.5	+0.2 [-0.8, +1.2]	+0.0 [-1.2, +1.2]
OLMo-2-1B-I	Values + names	24.7	25.7	25.2	-0.2 [-2.8, +2.5]	-1.3 [-3.8, +1.2]
OLMo-2-1B-I	Values only	18.8	19.8	18.3	-2.7 [-5.0, -0.5]	-1.7 [-4.0, +0.7]

Table 22: Numeric-equation and name-repetition controls. Queried/intermediate columns rename that variable. Added errors subtract the ordinary twin's suggested-digit rate (percentage points). Parse failures remain in denominators; coverage is 99.2–100% (rounded).

Model	Query name	Answer format	Pairs	Ordinary correct	Misleading correct	Added errors [95%]
Llama-3.2-3B	Omitted	Digit	600	99.7	99.8	+0.0 [+0.0, +0.0]
Llama-3.2-3B	Omitted	Assignment	600	100.0	100.0	+0.0 [+0.0, +0.0]
Llama-3.2-3B	Repeated	Digit	600	100.0	100.0	+0.0 [+0.0, +0.0]
Llama-3.2-3B	Repeated	Assignment	600	100.0	100.0	+0.0 [+0.0, +0.0]
Llama-3.1-8B	Omitted	Digit	600	100.0	100.0	+0.0 [+0.0, +0.0]
Llama-3.1-8B	Omitted	Assignment	600	100.0	100.0	+0.0 [+0.0, +0.0]
Llama-3.1-8B	Repeated	Digit	600	100.0	100.0	+0.0 [+0.0, +0.0]
Llama-3.1-8B	Repeated	Assignment	600	100.0	100.0	+0.0 [+0.0, +0.0]
OLMo-2-1B-I	Omitted	Digit	368	78.0	69.0	+12.8 [+9.1, +16.6]
OLMo-2-1B-I	Omitted	Assignment	368	94.0	94.3	+0.0 [+0.0, +0.0]
OLMo-2-1B-I	Repeated	Digit	368	96.2	94.3	+0.5 [+0.0, +1.4]
OLMo-2-1B-I	Repeated	Assignment	368	96.2	96.5	+0.0 [+0.0, +0.0]

Table 23: Final answers after fixed, completely correct calculations, with the misleading name on the queried variable. Correct answers are percentages; added errors are percentage points. All continuations parse. Assignment outputs include the queried name before the generated digit.

disruption control (Zhang and Nanda, 2024). The alternative-name control is uninformative in the main cell: every sum-family identifier implies the same sum, so there is no distinct source name-suggested value. Llama-3.1-8B's affected two-token name lacks aligned neutral twins for the name-site intervention.

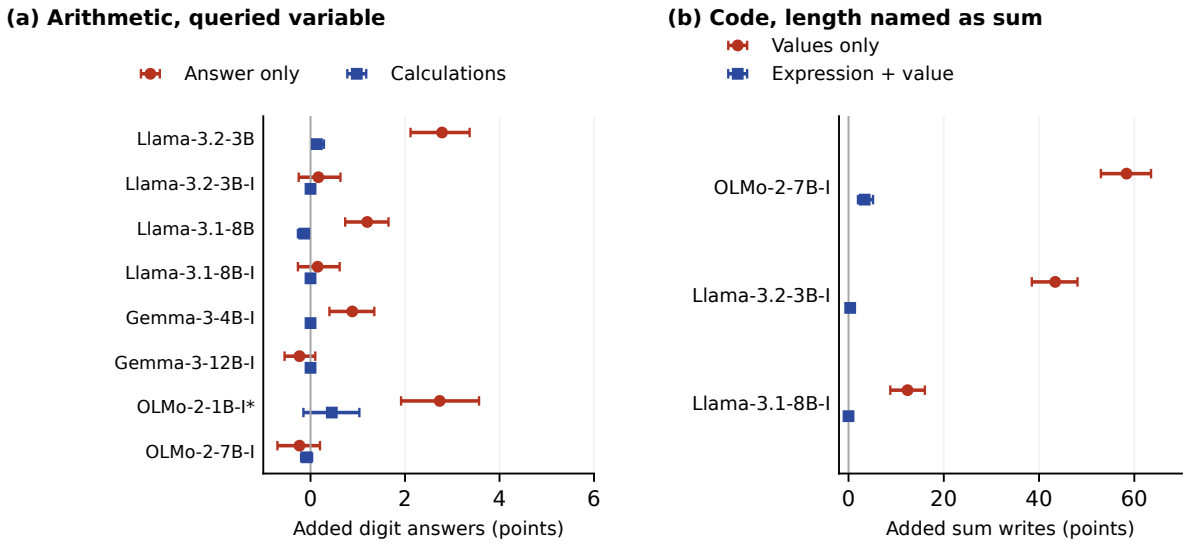
O Scope checks

Operation and scale. Moving the misleading name to a unary middle step produces no reliable effect for models above the accuracy floor; the largest excess is +0.6 points. CodeLlama-34B's length-to-sum effect is +0.0, while its maximum-to-sum effect is +1.8 points [+0.7, +3.1]. The latter is a reliable positive effect, but its interval extends beyond the two-point margin. Its small point estimate cannot be reported as equivalence. The larger-checkpoint result supports the registered maximum-to-sum direction and reliable-effect criterion, while its magnitude is much smaller than CodeLlama-7B's.

Natural code. We select 89 CRUXEval functions with length, loop-counter or maximum assignments, rename the target by AST rewriting to `v` or `total`, and verify execution after renaming. Direct and prose accuracy contrasts all have intervals covering zero on the 84 functions whose renamed target is used downstream. Their endpoints stay within 10 and 13 points, respectively (Table 32). These are bounded results for those functions and prompts; they do not test the trace that lists values without expressions used in the synthetic study. The natural-code functions come from Gu et al. (2024).

P Effect and equivalence checks

Each contrast subtracts the neutral twin's rate for the same digit. We resample matched problems with demonstration-seed replicates together and use 95% cluster-bootstrap intervals. Reliability



I: instruction-tuned. *Calculation accuracy below 90%.

Figure 8: Selected name-error contrasts (95% intervals); scales differ.

Level	Neutral program (body)	Traced steps
1	`v = len(xs); return v`	1
2	`v = len(xs); w = v * 2; return w`	2
3	`v = len(xs); w = max(xs); u = v + w; return u`	3
4	level 3 with an unused fourth assignment	3
5	`v = len(xs); w = v * 2; u = w + 3; return u`	3
6	level 5 with the misleading name on the middle step	3

Table 24: The levels, one example each. The aggregate in the first statement is any of `len`, `sum`, `max`, `min`; unary operations are `*2`, `//2`, `+c`, `-c`. The misleading name sits on the first statement at levels 1–5 and on the second at level 6.

also requires a consistent sign across all three demonstration sets.

Equivalence applies the ±2-point margin to 90% intervals from the same cluster bootstrap (two one-sided 5% tests), using 2,000 draws. Across 192 tested contrasts, 150 intervals lie wholly inside the margin, 36 contrasts have reliable effects without meeting that bound, and 6 meet neither criterion. A small estimate or an interval covering zero does not by itself establish equivalence.

Q Generated-prefix audit and initial readouts

For 2565 of 2565 misleading-name generations, the token prefix through the marker before the first value exactly matches the cached trace. This verifies that the cached state at that marker also precedes the generated value. No target value token is included. These initial readouts select layers on the full neutral evaluation set; the main text instead uses independent cross-fitted selection (Appendix T). We split the generated writes into name-suggested values, correct values and other errors. Table 33 averages three probe seeds and pools demonstration sets 7, 11 and 13. Intervals resample matched programs with all their demonstration repeats (4,000 draws). These conditional readouts establish available information on observed errors; they do not establish its causal use.

R Sensitivity to identifier meanings

None of the three main-cell names (`total`, `sum_all`, `acc`) meets the originally planned 80% panel gate. Its filtered estimate is therefore unavailable, rather than zero. Table 34 retains the full-data estimate and reports each name and each model's own meaning gate. Gates use the original saved meaning checks. Intervals cluster matched programs across the three demonstration sets. This exploratory sensitivity does not replace the planned panel gate or remove the deviation

Family	Names	Implied operation
len	`count`, `n_items`, `length`	`len(xs)`
sum	`total`, `sum_all`, `acc`	`sum(xs)`
max	`largest`, `max_val`, `peak`	`max(xs)`
min	`smallest`, `min_val`, `low`	`min(xs)`
double	`double`, `twice`	`2 * a`
half	`half`	`a // 2`
succ	`succ`, `next_val`	`a + 1`
pred	`pred`, `prev_val`	`a - 1`
add	`combined`, `both`	`a + b`
sub	`diff`, `gap`	`a - b`
mul	`product`, `scaled`	`a * b`

Table 25: Identifier families and their suggested operations. The gate (Appendix K) scores each name by the log-probability each model assigns to the candidate right-hand sides; 12 of 25 names are read as intended by at least 80% of models. The min and max names cannot produce name errors on the aggregate step (their value is an input element) and are used as congruent and alternative-name names.

Model	Return acc.	Direct: errors	Trace: len→sum	Trace: max→sum
OLMo-2-7B	95.7	+0.4	+58.4* [+53.0, +63.5]	+1.9 [+1.1, +3.0]
Llama-3.2-3B	97.4	+0.8	+43.4* [+38.5, +48.1]	+0.0 [+0.0, +0.0]
Llama-3.1-8B	99.6	+0.5	+12.4* [+8.8, +16.0]	+0.0 [+0.0, +0.0]
CodeLlama-7B	91.7	+1.0	+3.7* [+2.2, +5.4]	+10.7* [+7.9, +13.7]
CodeGemma-7B	99.2	+1.5*	+0.1 [+0.0, +0.4]	+0.0 [+0.0, +0.0]
Gemma-3-4B	99.1	+0.8	+0.0 [+0.0, +0.0]	+0.0 [+0.0, +0.0]
OLMo-2-1B	37.8	+3.1*	+3.7* [+2.2, +5.5]	+4.2* [+2.6, +5.9]
CodeLlama-34B	98.9	–	+0.0 [+0.0, +0.0]	+1.8* [+0.7, +3.1]

Table 26: Full level-5 panel: ordinary-name final-answer accuracy (%) and excess name errors (points). Direct contrasts rename the returned variable; trace contrasts rename the first assignment. Trace columns include 95% intervals. Stars mark consistent signs and intervals excluding zero.

from it.

S Length and numeric elaboration controls

We add neutral annotations to each demonstrated value, matching the expression demonstrations' token counts, and separately add valid numeric elaborations of the form `name = value + 0 = value`. The test program and `Trace:` prefix are unchanged. Greedy decoding uses the same 285 misleading programs and neutral twins for each format. Table 35 pools three demonstration sets; changes relative to the original trace and expression trace are paired on the same programs. The length-matched annotation controls retain much larger excess than expression demonstrations in both models. Numeric elaboration reduces excess relative to the original trace but leaves a larger effect than source expressions. Token count alone does not reproduce the expression benefit, and adding valid arithmetic accounts for part of it; these two contrasts do not isolate a single mechanism.

T Independent probe layer selection

The main-text error readouts use five-fold cross-fitting. A fixed SHA256 hash assigns canonical program computations to folds, keeping renamed versions and demonstration repeats together. For each held-out fold, we select the layer with the highest neutral accuracy on the other four folds, averaging three probe seeds and breaking ties toward the lowest layer. Neither the held-out neutral labels nor its misleading writes enter that choice. Probe training remains disjoint from every evaluation computation. Each program is scored only with its held-out fold's layer. The generated-prefix audit from Appendix Q still applies. Table 36 reports the conditional write strata and paired changes from the earlier full-neutral selection. Intervals use 4,000 canonical-program bootstrap draws, keeping demonstration repeats together. They condition on the fixed fold selectors and do not refit layer selection inside each bootstrap. Name controls and the full layer curves remain descriptive measurements in Appendix N.

Model	len→sum	max→sum	min→sum	sum→len	max→len
OLMo-2-7B	+17.6* / +58.4*	+0.9 / +1.9	+0.0 / +0.2	+4.4* / +0.0	+0.0 / +0.0
Llama-3.2-3B	+23.6* / +43.4*	+1.1 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Llama-3.1-8B	+1.5 / +12.4*	+0.7 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
CodeLlama-7B	+0.9 / +3.7*	+7.5 / +10.7*	+1.0 / +1.7	+1.5 / +0.0	+0.9 / +0.6
CodeGemma-7B	+0.0 / +0.1	+10.8* / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
Gemma-3-4B	+0.0 / +0.0	+0.2 / +0.0	+0.0 / +0.0	+0.0 / +0.0	+0.0 / +0.0
OLMo-2-1B	+1.2 / +3.7*	+2.8* / +4.2*	+0.3 / -0.2	+10.6* / +11.1*	+9.6* / +5.6*

Table 27: Excess name errors at the aggregate step, trace regime, level 3 / level 5, three sets of worked examples pooled; * as in Table 26.

Example format	Shown value	OLMo-2-7B-I	Llama-3.2-3B-I	Llama-3.1-8B-I
Values only	`v = 2`	+58.4	+43.4	+12.4
Matched descriptive text	`v = 2 [notes]`	+56.6	+49.6	–
Numeric elaboration	`v = 2 + 0 = 2`	+41.3	+14.9	–
Interactive Python	`>> v  2`	+9.5	+12.6	+0.2
Code comments	`v = len(xs) # 2`	+0.0	+6.8	+0.1
Expression and value	`v = len(xs) = 2`	+3.4	+0.4	+0.0

Table 28: Code format comparison: added sum writes (points). Dashes: untested.

U Independent replication with screened names

We fix 16 fresh sum-family candidates before scoring them with the original candidate-continuation meaning check (Appendix K). The panel is the same seven models used for that gate. A name qualifies when sum is the highest-scoring continuation in at least 80% of the panel; ties follow the original scoring order. Two additional function-name contexts and neutral names are diagnostic and do not affect selection. Of the candidates, 14 qualify. We freeze the first 3 qualifying names in the fixed order: `sum_of_values`, `sum_of_items`, `sum_of_elements`. No trace outcomes enter selection. The replication uses 200 new length-computed programs. Their canonical computations are unique and absent from the original level-5 training and test corpus and all three demonstration sets. True and name-suggested values are checked by execution and retain the original value-exclusion rules. Every selected name is crossed with every program; its neutral twin differs only in the target identifier. The three affected models and the prespecified CodeGemma-7B control use the same programs and demonstration sets 7, 11 and 13 under trace and expression demonstrations. Table 38 pools names and demonstration sets, clustering all repeats of each computation in 4,000 bootstrap draws. Table 39 reports each identifier. The format contrast replicates on independently selected names, but names and programs differ from the original experiment, so differences in effect magnitude across the two samples are descriptive. This exploratory replication does not retroactively repair the original selection-rule deviation. Probes and patches concern the original identifiers, not these new names.

V Prompt-end versus pre-write readouts

We fit separate probes at the final token of the complete prompt ending in `Trace:`, before generating any trace token. Tokenization of this prefix is checked against the original continuation. Training computations, demonstration sets, SGD recipe and seeds match the pre-write probes. Each position selects its layer independently using the same five canonical-computation folds and other folds' neutral labels. Outcome strata and generated-prefix eligibility are frozen from the previous audit. Intervals use 4,000 paired canonical-computation bootstrap draws and condition on the fixed selectors.

W Reproducibility and model terms

The main panel contains seven instruction-tuned checkpoints, with CodeLlama-34B as a scale check. We use CodeLlama (Rozière et al., 2023), CodeGemma (CodeGemma Team, 2024), Llama 3 (Grattafiori et al., 2024), Gemma 3 (Gemma Team, 2025) and OLMo 2 (Team OLMo, 2024; Walsh et al., 2025). The model identifiers are `codellama/CodeLlama-7b-Instruct-hf` and its 34b variant, `google/codegemma-7b-it`, `meta-llama/Llama-3.2-3B-Instruct`,

Format shown in examples	Example	OLMo-2-7B	Llama-3.2-3B	Llama-3.1-8B
Values only	`v = 2` `>>> v`	+58.4* [+53.0, +63.5]	+43.4* [+38.5, +48.1]	+12.4* [+8.8, +16.0]
Interactive Python transcript	`2`	+9.5* [+6.9, +12.0]	+12.6* [+9.4, +15.9]	+0.2 [+0.0, +0.6]
Code with value comments	`v = len(xs) # 2`	+0.0 [+0.0, +0.0]	+6.8* [+4.4, +9.2]	+0.1 [+0.0, +0.4]
Expression and value	`v = len(xs) = 2`	+3.4* [+2.0, +5.1]	+0.4 [+0.0, +0.9]	+0.0 [+0.0, +0.0]

Table 29: Baseline formats with 95% bootstrap intervals over matched problems.

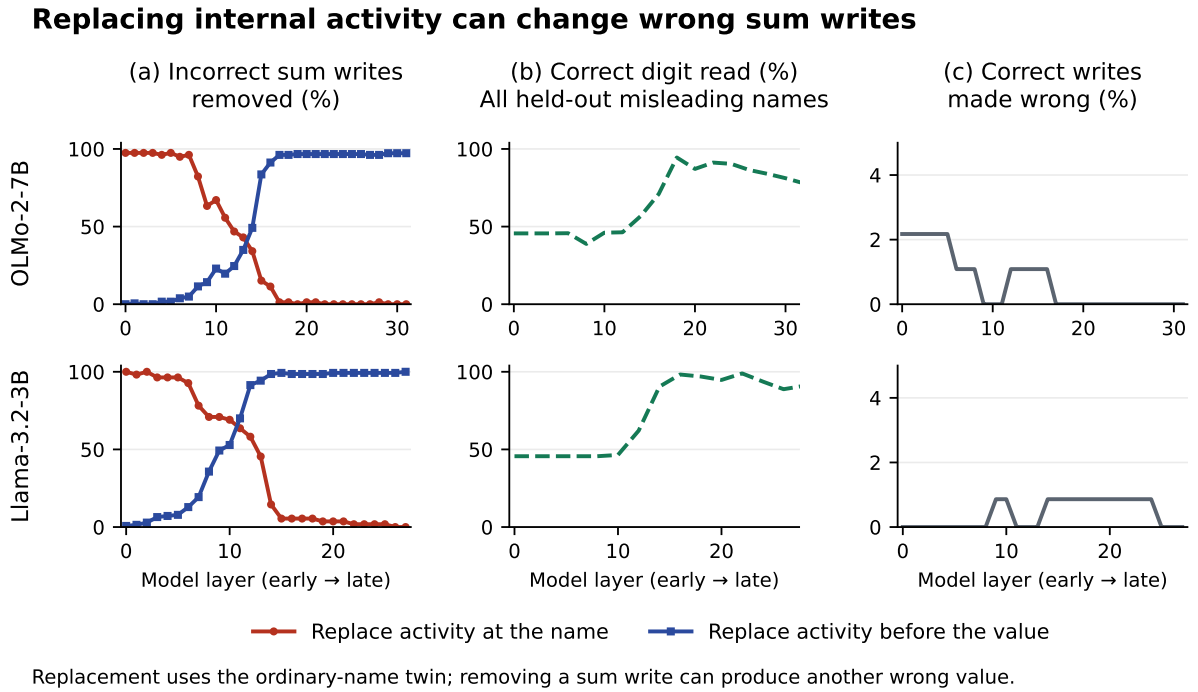


Figure 9: **(a)** Name-error removal. **(b)** Correct-digit readout. **(c)** Damage to correct writes. Patches use ordinary-name twins.

`meta-llama/Llama-3.1-8B-Instruct`, `google/gemma-3-4b-it`, `allenai/OLMo-2-0425-1B-Instruct` and `allenai/OLMo-2-1124-7B-Instruct`. These use the Llama 2/CodeLlama, Llama 3.1 or 3.2 Community Licenses, Gemma Terms of Use, and Apache 2.0, respectively. The study uses their released weights for research inference and does not redistribute weights. CRUXEval uses the MIT license.

Synthetic data consist of digits, generated identifiers and short Python functions, without personal records or human annotations. Each tested level has 1,000 matched problems per target. Main behavior uses three sets of three demonstrations with seeds 7, 11 and 13; programs and demonstration identities remain fixed across compared formats. Greedy decoding uses the full vocabulary. Parsers retain failed outputs in denominators; trace-error measurements use the first assignment, while final-answer accuracy scores the reported return value. Probe training and testing are disjoint in arithmetic computation, ignoring identifier spelling. The independent-name replication uses separate screened names and 200 new computations, as described in Appendix U.

Probes use full-batch SGD with learning rate 10^{-3} for 10,000 epochs and seeds 0, 1 and 2. Five-fold layer selection is disjoint in computation; bootstrap intervals condition on those selectors. These are linear-predictor fits, not language-model training. Combined recorded allocations across the arithmetic and code experiments total 222.5 GPU-hours on NVIDIA A30, H100 and H200 GPUs, an upper bound including idle and shared workers. Software is PyTorch 2.11.0 (CUDA 12.9) (Paszke et al., 2019), Transformers 5.14.1 (Wolf et al., 2020), NumPy 2.3.5 and scikit-learn 1.9.1 (Pedregosa et al., 2011).

Model	Set	Layer	Neutral	Control	Misleading	Control	Majority
OLMo-2-7B	7	18	99.5	100.0	94.5	33.0	45.6
OLMo-2-7B	11	18	97.3	100.0	95.4	0.0	45.6
OLMo-2-7B	13	18	99.8	100.0	97.4	23.4	45.6
Llama-3.2-3B	7	24	98.8	99.7	93.7	20.0	45.6
Llama-3.2-3B	11	20	100.0	99.8	95.4	5.0	45.6
Llama-3.2-3B	13	28	99.4	100.0	99.9	1.9	45.6
Llama-3.1-8B	7	16	100.0	87.8	99.9	33.0	45.6
Llama-3.1-8B	11	20	100.0	81.6	100.0	0.1	45.6
Llama-3.1-8B	13	20	100.0	87.2	100.0	3.0	45.6

Table 30: Disjoint code probes, three demonstration sets. Neutral and misleading are true-digit accuracy; each control column gives name-identity accuracy for that condition. Majority predicts the most frequent digit in training on the misleading test programs. Rates are percentages, averaged over three probe seeds. Layers are selected by neutral accuracy.

	Name tokens			Decision token			
Model	L0	damage	<10% by	50% at	90% at	all layers	damage
OLMo-2-7B	97%	2%	L17	L15	L16	97%	2%
Llama-3.2-3B	100%	0%	L15	L10	L12	100%	0%
Llama-3.1-8B	–	–	–	L8	L9	100%	0%

Table 31: Layers behind Figure 9. Name tokens: name errors removed by replacing every token of the name at layer 0, damage to correct instances, and the first layer after which removal falls below 10%. Decision token: first layer at which a single-layer replacement removes 50% and 90% of name errors, removal with every layer replaced, and its damage. Llama-3.1-8B's name errors fall on a two-token name without a token-aligned twin.

X Correctness on the matched code format cohort

We score the first written value and final answer on the same 285 original length-to-sum programs and their ordinary-name twins, under values-only and expression-and-value examples. Each format has 855 paired observations over three demonstration sets. Parse failures count as incorrect. Intervals use 4,000 bootstrap draws over matched program IDs, keeping all demonstration repeats and both names together. The format changes are paired on those same IDs. These are overall correctness rates within the specified cohort, rather than the larger task pool. Zero-width intervals record this sample, not certainty about population rates.

Model	Direct acc.	Direct Δ	Prose acc.	Prose Δ
OLMo-2-7B	10	+1.2 [+0.0, +3.6]	22	-1.2 [-9.5, +8.3]
Llama-3.2-3B	19	+1.2 [-2.4, +4.8]	25	+3.6 [-3.6, +10.7]
Llama-3.1-8B	22	-3.6 [-9.5, +2.4]	40	+0.0 [-9.5, +9.5]
CodeLlama-7B	18	+2.4 [+0.0, +6.0]	13	+2.4 [-4.8, +9.5]
CodeGemma-7B	18	+0.0 [-4.8, +4.8]	22	+2.4 [-8.3, +13.1]
Gemma-3-4B	15	-2.4 [-6.0, +0.0]	45	+4.8 [-1.2, +11.9]
OLMo-2-1B	16	-3.6 [-8.3, +0.0]	3	+6.0 [+0.0, +11.9]

Table 32: 89 CRUXEval functions that assign a variable from `len(...)` (48), a loop counter (38) or `max(...)` (3), of which 84 use it downstream; each rendered with the original name, the neutral name `v`, and the misleading name `total`. Accuracy on all neutral renderings, and the paired misleading-minus-neutral difference in points on the downstream-used subset, with a bootstrap interval over functions.

Model	Generated write	Rows	Programs	True-digit accuracy (%)
OLMo-2-7B	name error	503	193	96.4 [94.8, 97.8]
OLMo-2-7B	correct write	320	135	96.7 [94.4, 98.5]
OLMo-2-7B	other error	32	24	77.1 [61.7, 90.5]
Llama-3.2-3B	name error	372	169	95.7 [93.8, 97.4]
Llama-3.2-3B	correct write	440	204	98.3 [97.0, 99.3]
Llama-3.2-3B	other error	43	29	82.2 [71.2, 91.7]
Llama-3.1-8B	name error	106	45	99.7 [99.0, 100.0]
Llama-3.1-8B	correct write	749	261	100.0 [100.0, 100.0]
Llama-3.1-8B	other error	0	0	–

Table 33: Disjoint probes at the verified generated prefix. A dash denotes an empty stratum. The denominator is observed writes within each stratum, not independent demonstration repeats.

Model	All	Model gate	`total`	`sum_all`	`acc`
Gemma-3-4B	0.0 [0.0, 0.0]	–	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
CodeGemma-7B	0.1 [0.0, 0.4]	0.1 [0.0, 0.4]	0.0 [0.0, 0.0]	0.3 [0.0, 1.0]	0.0 [0.0, 0.0]
Llama-3.1-8B	12.4 [8.9, 16.0]	12.4 [8.9, 16.0]	0.0 [0.0, 0.0]	34.0 [26.0, 42.3]	0.0 [0.0, 0.0]
OLMo-2-1B	3.7 [2.2, 5.5]	5.0 [1.8, 8.9]	5.0 [1.8, 8.9]	5.8 [2.6, 9.3]	0.0 [0.0, 0.0]
OLMo-2-7B	58.4 [53.1, 63.4]	59.3 [52.7, 65.8]	56.4 [47.9, 64.9]	96.2 [93.9, 98.1]	15.3 [9.6, 21.8]
CodeLlama-7B	3.7 [2.1, 5.5]	3.7 [2.1, 5.5]	1.8 [0.4, 3.2]	8.7 [4.5, 13.1]	0.0 [0.0, 0.0]
Llama-3.2-3B	43.4 [38.5, 48.0]	43.4 [38.5, 48.0]	30.1 [23.8, 36.9]	76.3 [70.2, 82.4]	18.4 [11.9, 25.7]

Table 34: Excess name errors in percentage points [95% interval], level 5 length-to-sum. Empty subsets are shown as dashes. The saved JSON also reports each demonstration set and the all-three-sets sign rule.

Model	Format	Excess	Change vs trace	Change vs expression
OLMo-2-7B	neutral annotation	56.6 [51.3, 61.6]	-1.8 [-3.4, -0.1]	53.2 [48.1, 58.2]
OLMo-2-7B	numeric elaboration	41.3 [36.4, 46.0]	-17.1 [-21.1, -12.9]	37.9 [33.2, 42.5]
Llama-3.2-3B	neutral annotation	49.6 [44.7, 54.3]	6.2 [4.1, 8.4]	49.2 [44.3, 53.9]
Llama-3.2-3B	numeric elaboration	14.9 [12.0, 17.8]	-28.5 [-32.7, -24.3]	14.5 [11.7, 17.4]

Table 35: Excess name errors and paired changes in percentage points [95% interval]. Bootstrap clusters keep all demonstration repeats for each matched program together. These controls assess length and numeric elaboration; they do not by themselves establish how expressions change generation.

Model	Write	Rows	Computations	Accuracy (%)	Change (points)
OLMo-2-7B	name error	503	193	96.4 [94.9, 97.9]	0.0 [0.0, 0.0]
OLMo-2-7B	correct write	320	135	96.7 [94.4, 98.4]	0.0 [0.0, 0.0]
OLMo-2-7B	other error	32	24	77.1 [61.6, 90.6]	0.0 [0.0, 0.0]
Llama-3.2-3B	name error	372	169	95.5 [93.6, 97.3]	-0.2 [-0.9, 0.4]
Llama-3.2-3B	correct write	440	204	98.0 [96.6, 99.1]	-0.3 [-0.8, 0.0]
Llama-3.2-3B	other error	43	29	82.2 [71.1, 91.7]	0.0 [0.0, 0.0]
Llama-3.1-8B	name error	106	45	99.7 [99.0, 100.0]	0.0 [0.0, 0.0]
Llama-3.1-8B	correct write	749	261	100.0 [99.9, 100.0]	-0.0 [-0.1, 0.0]
Llama-3.1-8B	other error	0	0	–	–

Table 36: True-digit readouts with independent layer selection. Changes compare the same observed writes with the earlier full-neutral layer choice. A dash denotes an empty stratum. These readouts establish decodability, not causal use.

Candidate	Sum agreement	Qualifies	Selected
`sum_of_values`	6/7	yes	yes
`sum_of_items`	7/7	yes	yes
`sum_of_elements`	7/7	yes	yes
`sum_of_xs`	7/7	yes	no
`sum_of_list`	6/7	yes	no
`list_sum`	6/7	yes	no
`input_sum`	6/7	yes	no
`items_sum`	6/7	yes	no
`elements_sum`	5/7	no	no
`sum_values`	5/7	no	no
`sum_result`	6/7	yes	no
`sum_total`	6/7	yes	no
`total_sum`	6/7	yes	no
`xs_sum`	7/7	yes	no
`sum_xs`	7/7	yes	no
`summation`	6/7	yes	no

Table 37: The entire fixed candidate list, in selection order. Qualification uses the original canonical meaning prompt. Selection retains the first three qualifying candidates without consulting behavior.

Model	Trace excess	Expression excess	Expression minus trace
OLMo-2-7B	91.1 [88.4, 93.6]	0.8 [0.3, 1.3]	-90.3 [-92.9, -87.6]
Llama-3.2-3B	61.5 [56.1, 66.9]	0.0 [0.0, 0.0]	-61.5 [-66.9, -56.1]
Llama-3.1-8B	16.7 [12.8, 20.8]	0.0 [0.0, 0.0]	-16.7 [-20.8, -12.8]
CodeGemma-7B	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]

Table 38: Independent screened-name replication: excess name errors and paired format changes in percentage points [95% interval]. All selected identifiers and three demonstration sets are pooled over 200 fresh computations. A zero bootstrap interval records no observed paired name errors; it is not a population-level absence or equivalence claim.

Model	Identifier	Trace excess	Expression excess
OLMo-2-7B	`sum_of_values`	91.7 [88.8, 94.3]	0.2 [0.0, 0.5]
OLMo-2-7B	`sum_of_items`	89.2 [86.0, 92.0]	2.0 [0.8, 3.3]
OLMo-2-7B	`sum_of_elements`	92.5 [89.8, 94.8]	0.2 [0.0, 0.5]
Llama-3.2-3B	`sum_of_values`	61.7 [56.0, 67.3]	0.0 [0.0, 0.0]
Llama-3.2-3B	`sum_of_items`	55.5 [50.0, 61.3]	0.0 [0.0, 0.0]
Llama-3.2-3B	`sum_of_elements`	67.3 [61.7, 73.0]	0.0 [0.0, 0.0]
Llama-3.1-8B	`sum_of_values`	12.8 [9.3, 16.7]	0.0 [0.0, 0.0]
Llama-3.1-8B	`sum_of_items`	22.8 [18.0, 27.7]	0.0 [0.0, 0.0]
Llama-3.1-8B	`sum_of_elements`	14.5 [10.5, 18.7]	0.0 [0.0, 0.0]
CodeGemma-7B	`sum_of_values`	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
CodeGemma-7B	`sum_of_items`	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
CodeGemma-7B	`sum_of_elements`	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]

Table 39: Per-identifier replication, pooling the same three demonstration sets. Effects are percentage points [95% canonical-program bootstrap interval]. All affected-model trace means are positive in every demonstration set; per-set estimates are retained in the saved summary.

Model	Write	Rows / computations	Prompt	Before write	Change	Name ID
OLMo-2-7B	sum write	503 / 193	38.4 [32.9, 43.9]	96.4 [94.9, 97.9]	+58.1 [+52.6, +63.4]	12.7 [9.8, 15.9]
OLMo-2-7B	correct write	320 / 135	35.2 [27.7, 43.1]	96.7 [94.4, 98.4]	+61.5 [+53.3, +69.3]	20.8 [15.5, 26.3]
OLMo-2-7B	other error	32 / 24	76.0 [58.1, 90.2]	77.1 [61.6, 90.6]	+1.0 [-19.6, +22.6]	26.0 [12.6, 41.1]
Llama-3.2-3B	sum write	372 / 169	26.4 [22.4, 30.8]	95.5 [93.6, 97.3]	+69.1 [+64.5, +73.4]	0.7 [0.1, 1.6]
Llama-3.2-3B	correct write	440 / 204	37.9 [32.1, 43.7]	98.0 [96.6, 99.1]	+60.1 [+54.4, +65.8]	3.6 [1.8, 5.7]
Llama-3.2-3B	other error	43 / 29	20.9 [11.7, 31.7]	82.2 [71.1, 91.7]	+61.2 [+46.8, +73.8]	2.3 [0.0, 6.1]
Llama-3.1-8B	sum write	106 / 45	56.0 [47.4, 64.2]	99.7 [99.0, 100.0]	+43.7 [+35.4, +52.4]	0.0 [0.0, 0.0]
Llama-3.1-8B	correct write	749 / 261	49.9 [45.7, 54.1]	100.0 [99.9, 100.0]	+50.1 [+45.8, +54.3]	0.0 [0.0, 0.0]
Llama-3.1-8B	other error	0 / 0	–	–	–	–

Table 40: Paired true-digit accuracies and prompt-position name-identity controls (%) [95% interval]. Change is pre-write minus prompt accuracy in points.

Model	Extra sum writes (points)	Correct digit (%) Before trace	Correct digit (%) Before wrong value	Wrong writes (unique programs)
OLMo-2-7B	+58.4	38.4 [32.9, 43.9]	96.4 [94.9, 97.9]	503 (193)
Llama-3.2-3B	+43.4	26.4 [22.4, 30.8]	95.5 [93.6, 97.3]	372 (169)
Llama-3.1-8B	+12.4	56.0 [47.4, 64.2]	99.7 [99.0, 100.0]	106 (45)

Table 41: Code behavior and error-conditioned readouts [95% interval]. Counts distinguish writes from computations.

Model	Examples	Names	Correct first value (%)	Correct final answer (%)
OLMo-2-7B-I	Values only	Ordinary	95.0 [93.3, 96.5]	93.1 [91.1, 95.0]
OLMo-2-7B-I	Values only	Misleading	37.4 [32.6, 42.6]	37.1 [32.2, 42.2]
OLMo-2-7B-I	Expression + value	Ordinary	96.5 [95.1, 97.8]	96.6 [95.3, 97.9]
OLMo-2-7B-I	Expression + value	Misleading	92.0 [89.8, 94.2]	92.6 [90.3, 94.7]
Llama-3.2-3B-I	Values only	Ordinary	98.9 [98.2, 99.5]	95.6 [93.9, 97.1]
Llama-3.2-3B-I	Values only	Misleading	51.5 [46.9, 56.3]	51.0 [46.3, 55.8]
Llama-3.2-3B-I	Expression + value	Ordinary	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Llama-3.2-3B-I	Expression + value	Misleading	99.3 [98.6, 99.9]	99.4 [98.7, 99.9]
Llama-3.1-8B-I	Values only	Ordinary	100.0 [100.0, 100.0]	99.6 [99.1, 100.0]
Llama-3.1-8B-I	Values only	Misleading	87.6 [84.0, 91.1]	87.7 [84.1, 91.2]
Llama-3.1-8B-I	Expression + value	Ordinary	100.0 [100.0, 100.0]	99.4 [98.5, 100.0]
Llama-3.1-8B-I	Expression + value	Misleading	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]
Expression minus values-only correctness (percentage points)				
OLMo-2-7B-I	Paired change	Ordinary	1.5 [0.2, 2.8]	3.5 [1.8, 5.4]
OLMo-2-7B-I	Paired change	Misleading	54.6 [49.6, 59.4]	55.6 [50.3, 60.5]
Llama-3.2-3B-I	Paired change	Ordinary	1.1 [0.5, 1.8]	4.4 [2.9, 6.1]
Llama-3.2-3B-I	Paired change	Misleading	47.8 [43.2, 52.4]	48.4 [43.6, 53.1]
Llama-3.1-8B-I	Paired change	Ordinary	0.0 [0.0, 0.0]	-0.2 [-1.3, 0.7]
Llama-3.1-8B-I	Paired change	Misleading	12.4 [8.9, 16.0]	12.3 [8.8, 15.9]

Table 42: Matched-code correctness and paired format changes (95% intervals).